

Plateforme IoT Maison Intelligente

Système Complet d'Automatisation Résidentielle Intelligente

Rapport de Projet de Fin d'Études (PFE)

Licence Appliquée en Informatique

INFORMATIONS

Réalisé par : Imen Khadhrawi

: Malak Hammami

Encadrant académique : [Nom de l'Encadrant]

Encadrant industriel : [Nom du Tuteur Industriel]

Année académique : 2025 – 2026

Node.js · React · MongoDB · MQTT · ESP32 · Socket.IO · Groq AI

Dédicace

À ma famille, dont le soutien inconditionnel a rendu ce voyage possible.

*À mes professeurs et mentors à l'ISSET Siliana,
dont les conseils ont forgé mon état d'esprit d'ingénieur.*

*À tous les développeurs qui croient que la technologie
peut rendre la vie quotidienne plus sûre et plus intelligente.*

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadrant académique pour ses conseils continus et ses retours perspicaces tout au long de ce projet. Je suis également reconnaissant envers mon tuteur industriel pour m'avoir fourni une perspective concrète sur les systèmes de maison intelligente et l'architecture IoT.

Je remercie chaleureusement le corps enseignant de l'ISET Siliana pour les solides fondations théoriques et pratiques fournies tout au long du cursus. Les cours en systèmes distribués, génie logiciel et systèmes embarqués ont été directement déterminants dans la conception et la mise en œuvre de cette plateforme.

Je remercie également mes collègues et amis qui ont participé aux sessions de test, signalé des bogues et suggéré des améliorations. Leurs retours pratiques ont considérablement amélioré l'expérience utilisateur du produit final.

Enfin, merci à la communauté open source derrière Node.js, React, MongoDB, Aedes MQTT et toutes les bibliothèques utilisées dans ce projet.

Résumé

Les systèmes de maison intelligente sont de plus en plus intégraux à la vie résidentielle moderne, mais la plupart des solutions commerciales restent fragmentées entre différentes applications, protocoles et fournisseurs. Ce rapport de PFE présente la **Plateforme IoT Maison Intelligente**, un système unifié, full-stack et de qualité production développé selon la méthodologie agile Scrum en six sprints.

La plateforme fournit un backend **Node.js/Express** avec un courtier **Aedes MQTT** intégré, une application web progressive **React/Vite**, et un firmware **ESP32** pour l'intégration des dispositifs physiques. Les fonctionnalités clés comprennent : la gestion multi-rôles (Agence, Administrateur, Résident), le contrôle des appareils en temps réel via MQTT et Socket.IO, un assistant IA alimenté par l'API Groq, la gestion des urgences gaz avec contrôle matériel (vanne et alarme), une visualisation 3D isométrique du plan d'étage, la surveillance énergétique, l'automatisation par scénarios temporels, et un système complet de journalisation des audits.

Le système a été conçu selon les principes d'architecture en couches, de modélisation orientée domaine et de communication événementielle. Les fonctionnalités de sécurité comprennent l'authentification JWT, Google OAuth 2.0, l'authentification à deux facteurs TOTP, la limitation de débit et les middlewares d'autorisation basés sur les rôles.

Ce rapport documente le cycle de vie Scrum complet : définition du backlog produit, planification des sprints, modélisation UML, implémentation, tests et déploiement, en concluant par une revue de sécurité et une feuille de route future.

Mots-clés : IoT, Maison Intelligente, MQTT, Node.js, React, MongoDB, ESP32, Socket.IO, Scrum, Agile, JWT, Groq IA, Systèmes Temps Réel.

Abstract

Smart home systems are increasingly integral to modern residential life, yet most commercial solutions remain fragmented. This PFE report presents the **SmartHome IoT Platform**, a unified, full-stack home automation system built using the Scrum agile methodology over six sprints. The platform integrates Node.js/Express, Aedes MQTT, React/Vite, Socket.IO, MongoDB, ESP32 firmware, and Groq AI, delivering multi-role governance, real-time device control, gas safety emergency handling, 3D floorplan visualization, and scenario automation.

Keywords : IoT, Smart Home, MQTT, Node.js, React, MongoDB, ESP32, Scrum, JWT, Groq AI.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
1 Introduction Générale	1
1.1 Contexte et Motivation	1
1.2 Cadre du Projet	1
1.3 Problématique	2
1.4 Objectifs du Projet	2
1.5 Approche Méthodologique — Scrum	3
1.6 Contributions Scientifiques et Techniques	4
1.7 Organisation du Rapport	5
2 Analyse des Besoins et Backlog Produit	6
2.1 Parties Prenantes et Acteurs	6

2.2 Besoins Fonctionnels	7
2.2.1 Authentification et Gestion des Utilisateurs	7
2.2.2 Gestion des Appareils.....	7
2.2.3 Plan d'Étage et Visualisation	8
2.2.4 Systèmes de Sécurité	8
2.2.5 Assistant IA, Scénarios et Énergie.....	9
2.3 Besoins Non Fonctionnels.....	9
2.4 Backlog Produit.....	10
2.5 Diagramme de Cas d'Utilisation Global	13
3 État de l'Art et Stack Technologique	14
3.1 Panorama des Plateformes de Maison Intelligente	14
3.2 Protocoles de Communication IoT	15
3.2.1 MQTT (Message Queuing Telemetry Transport).....	15
3.2.2 Comparaison des Protocoles	16
3.3 Évaluation du Stack Technologique.....	16
3.3.1 Backend : Node.js et Express	16
3.3.2 Base de Données : MongoDB et Mongoose	16
3.3.3 Frontend : React et Vite	17
3.3.4 Firmware IoT : Arduino / ESP32	17
3.3.5 Intégration IA : API Groq	17
3.4 Environnement de Développement	18
3.5 Synthèse.....	18
4 Architecture Système et Modélisation UML	19
4.1 Vision Architecturale	19
4.2 Diagramme de Déploiement.....	20
4.3 Diagramme de Composants.....	21

4.4	Structure du Monorepo.....	21
4.5	Diagramme de Classes (Modèle Domaine)	22
4.6	Diagramme de Séquence — Commande d’Appareil.....	23
4.7	Diagramme de Séquence — Urgence Gaz	24
4.8	Machine à États — Mode Urgence	25
4.9	Flux de Données Bout en Bout	26
5	Sprint 1 & 2 — Authentification, Gestion des Utilisateurs et Contrôle des Appareils	27
5.1	Backlog Sprint 1	27
5.2	Modèle de Base de Données : Schéma Utilisateur	28
5.2.1	Algorithme de Génération du Code Maison.....	29
5.3	Flux d’Authentification.....	29
5.3.1	Inscription et Connexion Email/Mot de Passe	30
5.3.2	Authentification à Deux Facteurs TOTP	30
5.4	Middleware JWT	31
5.5	Revue et Rétrospective Sprint 1.....	32
5.6	Modèle Appareil	32
5.6.1	Types d’Appareils et Puissance Estimée.....	33
5.7	Cartographie GPIO de l’ESP32	33
5.8	Publication MQTT des Commandes.....	34
5.9	Revue et Rétrospective Sprint 2.....	35
6	Sprint 3 — Visualisation du Plan d’Étage, Contrôle de Scènes et Synchronisation Temps Réel	36
6.1	Modèle État Maison (HouseState).....	36
6.2	Routes API du Plan d’Étage.....	37
6.3	Implémentation des Scènes d’Éclairage	37

6.4	Passerelle Socket.IO Temps Réel	39
6.4.1	Authentification et Adhésion aux Chambres	39
6.4.2	Catalogue des Événements Socket.IO	39
6.5	Plan d'Étage 3D Isométrique	40
6.6	Revue et Rétrospective Sprint 3.	41
7	Sprint 4 — Détection Gaz, Capteur DHT11 et Protocoles d'Urgence	42

7.1	Service de Fond Gas Monitor	42
7.2	Calcul PPM du Capteur Gaz MQ6.....	43
7.3	Service DHT11 et Firmware	44
7.4	Overlay d'Urgence Frontend	45
7.5	Revue et Rétrospective Sprint 4.	45
8	Sprint 5 — Compagnon IA (Melo), Automatisation par Scénarios et Surveillance Énergétique	47

8.1	Backlog Sprint 5	47
8.2	Compagnon IA Melo	48
8.2.1	Architecture	48
8.2.2	Construction du Prompt Système	48
8.2.3	Implémentation de l'Endpoint Chat	49
8.2.4	Composant Frontend Melo	50
8.3	Modèle de Scénario et Moteur d'Automatisation	52
8.3.1	Schéma Scénario	52
8.3.2	Service Planificateur.....	52
8.4	Surveillance Énergétique.....	54
8.4.1	Algorithme de Calcul de la Consommation	54
8.5	Système de Demandes de Permission	55

8.5.1	Modèle DemandePermission	55
8.5.2	Flux d'Approbation de Permission	55
8.6	Revue et Rétrospective Sprint 5.....	56
9	Sprint 6 — Portail Agence, Journal d'Audit et Finalisation	57
9.1	Hub Conciergerie et Portail Agence	57
9.1.1	Server-Sent Events (SSE) pour le Statut en Direct	57
9.2	Système de Journal d'Audit.....	58
9.2.1	Schéma AuditLog	58
9.2.2	Événements Journalisés	58
9.3	Modes de Sécurité.....	59
9.4	Récapitulatif Complet des Routes API	60
9.5	Revue et Rétrospective Sprint 6.....	61
10	Sécurité, Tests et Assurance Qualité	62
10.1	Architecture de Sécurité	62
10.2	Conformité OWASP Top 10	62
10.3	Stratégie de Tests	63
10.3.1	Cas de Test Principaux	64
10.4	Limitations Connues	65
11	Gestion de Projet, Planification et Gouvernance des Risques	66
11.1	Mise en Œuvre du Cadre Scrum	66
11.1.1	Rôles et Cérémonies	66
11.1.2	Cérémonies de Sprint	66
11.2	Chronogramme du Projet.....	67

11.3	Vélocité des Sprints	67
11.4	Registre des Risques.....	68
11.5	Métriques Récapitulatives du Projet	69
12	Conclusion et Perspectives	70
12.1	Conclusion du Projet.....	70
12.1.1	Leçons Apprises.....	71
12.2	Perspectives et Travaux Futurs	71
12.2.1	Améliorations à Court Terme (3 prochains mois).....	71
12.2.2	Fonctionnalités à Moyen Terme (3–6 mois)	72
12.2.3	Directions de Recherche à Long Terme (6+ mois)	72
12.3	Remarques Finales	73
13	Annexe A : Référence Complète de l'API	74
13.1	Vue d'Ensemble.....	74
13.2	Endpoints d'Authentification (/api/auth).....	74
13.3	Endpoints Appareils (/api/devices)	75
13.4	Endpoints Plan d'Étage (/api/floorplan).....	75
13.5	Endpoints Sécurité Gaz (/api/gas).....	76
13.6	Endpoint Compagnon IA (/api/companion).....	76
13.7	Endpoints Scénarios (/api/scenarios)	76
13.8	Codes de Statut HTTP Courants	77
14	Annexe B : Architecture Frontend et Interface Utilisateur	78
14.1	Routage de l'Application	78
14.2	Inventaire des Pages.....	79
14.3	Composants Réutilisables Clés.....	80

14.4 Implémentation AuthContext	81
14.5 ThemeContext et Mode Sombre	82
15 Annexe C : Guide d'Installation et Runbook Opérationnel	84
15.1 Prérequis	84
15.2 Variables d'Environnement Backend	84
15.3 Étapes d'Installation Complètes	85
15.4 Configuration du Firmware ESP32	86
15.4.1 Installation des Bibliothèques Arduino IDE	86
15.4.2 Étapes de Flashage	87
15.5 Déploiement Production (Render / Railway)	87
15.6 Runbook Opérationnel	87
15.6.1 Vérifications de l'État des Services	88
15.6.2 Résolution des Problèmes Courants	88
16 Annexe D : Matrice de Traçabilité des Exigences	89
16.1 Objectif	89
16.2 Matrice de Traçabilité	89
16.3 Données de Burndown des Sprints	91
16.4 Verrouillage des Versions Technologiques	92

Table des figures

2.1	Diagramme de Cas d'Utilisation Global de la Plateforme SmartHome . . .	13
4.1	Diagramme de Déploiement de la Plateforme SmartHome	20
4.2	Diagramme de Composants du Backend	21
4.3	Diagramme de Classes du Modèle Domaine	22
4.4	Diagramme de Séquence — Flux de Commande d'Appareil	23
4.5	Diagramme de Séquence — Détection et Réponse à l'Urgence Gaz	24
4.6	Machine à États — Transitions du Mode Urgence Maison	25
4.7	Flux de Données de Commande d'Appareil Bout en Bout	26
5.1	Flux d'Inscription et de Connexion	30
10.1	Couches de Sécurité en Défense en Profondeur	62
11.1	Diagramme de Gantt du Projet (12 Semaines)	67

Liste des tableaux

1.1	Objectifs et Livrables du Projet	3
1.2	Résumé des Sprints	4
2.1	Acteurs Système et Responsabilités	6
2.2	Besoins Fonctionnels — Authentification	7
2.3	Besoins Fonctionnels — Gestion des Appareils	7
2.4	Besoins Fonctionnels — Plan d'Étage	8
2.5	Besoins Fonctionnels — Sécurité	8
2.6	Besoins Fonctionnels — IA, Scénarios et Énergie	9
2.7	Besoins Non Fonctionnels	10
2.8	Backlog Produit Complet	10
3.1	Comparaison des Solutions de Maison Intelligente	15
3.2	Caractéristiques Clés de MQTT	15
3.3	Bibliothèques Frontend Utilisées	17
5.1	Décomposition des User Stories — Sprint 1	27
5.2	Types d'Appareils et Consommation Estimée	33
5.3	Affectation des Broches GPIO ESP32	34
6.1	Routes API du Plan d'Étage	37
6.2	Référence des Événements Socket.IO	39
8.1	Décomposition des User Stories Sprint 5	47
9.1	Catégories et Événements du Journal d'Audit	59
9.2	Définition des Modes de Sécurité	59
9.3	Toutes les Routes Backend API	60

10.1 Conformité OWASP Top 10 (2021)	63
10.2 Stratégie de Tests par Niveau	63
10.3 Cas de Test d'Intégration Sélectionnés	64
10.4 Limitations Connues et Mitigations	65
11.1 Correspondance des Rôles Scrum	66
11.2 Suivi de la Vitesse par Sprint	68
11.3 Registre des Risques du Projet	68
13.1 API Authentification	74
13.2 API Appareils	75
13.3 API Plan d'Étage	75
13.4 API Gaz	76
13.5 API Compagnon	76
13.6 API Scénarios	77
14.1 Composants de Pages Frontend	79
14.2 Composants Réutilisables Clés	81
16.1 Matrice de Traçabilité des Exigences	89
16.2 Points de Story Restants par Burndown Sprint	92
16.3 Versions Exactes des Technologies Utilisées	92

Introduction Générale

1.1 Contexte et Motivation

Les maisons connectées ont évolué d'écosystèmes de gadgets isolés vers des systèmes cyber-physiques intégrés où la captation, l'actionnement, l'analyse et l'interaction utilisateur sont étroitement couplés. Selon Statista (2024), le nombre de dispositifs de maison intelligente en service dans le monde a dépassé 1,3 milliard d'unités, et le marché mondial de la maison intelligente est projeté à 622 milliards de dollars d'ici 2026.

Malgré cette croissance, la plupart des solutions disponibles restent fragmentées : une application mobile pour les caméras, une autre pour l'éclairage, et des outils distincts pour les dispositifs de sécurité. Cette fragmentation crée des inefficacités opérationnelles, affaiblit la posture de sécurité et génère des frictions pour les utilisateurs quotidiens.

La **Plateforme IoT Maison Intelligente** répond à cette lacune avec une solution unifiée : une plateforme unique de qualité production qui intègre la gestion des appareils, les canaux de commande et de télémétrie en temps réel, la gouvernance du foyer, les flux d'urgence sécuritaire, un assistant IA, et des scénarios d'automatisation extensibles — le tout accessible depuis une interface web responsive.

1.2 Cadre du Projet

Ce projet a été développé en tant que Projet de Fin d'Études (PFE) à **ISSET Siliana** — **Institut Supérieur des Etudes Technologiques de Siliana**, année académique 2025–2026, en vue de l'obtention de la Licence Appliquée en Informatique.

Le projet est organisé comme un **monorepo** contenant trois artefacts déployables :

- **Service backend** — API REST Node.js/Express, courtier MQTT Aedes intégré, passerelle Socket.IO, et couche de persistance MongoDB.
- **Client frontend** — Application monopage React 19/Vite avec routage conscient des rôles, synchronisation d'état en temps réel, plan d'étage 3D isométrique, et interface de chat IA.
- **Firmware ESP32** — Sketch Arduino sur microcontrôleur ESP32-D intégrant la détection de température/humidité (DHT11), la détection de gaz (MQ6), le contrôle de LEDs RGB, et l'actionnement de servomoteurs.

1.3 Problématique

Le problème d'ingénierie central traité par ce projet est la conception et la mise en œuvre d'une **plateforme multi-locataires de maison intelligente scalable** qui satisfait simultanément les contraintes suivantes :

1. **Réactivité temps réel** pour les événements utilisateur et la télémétrie IoT avec une latence bout en bout inférieure à 500 ms.
2. **Séparation claire des rôles** entre les comptes Agence, Administrateur de maison, Résident et SuperAdmin avec une gestion granulaire des permissions.
3. **Comportement critique de sécurité** pour les incidents gazeux avec des transitions d'état d'urgence déterministes, une actuation matérielle et une notification multi-canal.
4. **Architecture maintenable** supportant l'extension progressive des fonctionnalités sans rompre les contrats existants.
5. **Intégration IA accessible** permettant le contrôle des appareils en langage naturel via une API de modèle de langage à faible latence.

1.4 Objectifs du Projet

Les objectifs du projet, mappés à des livrables mesurables, sont les suivants :

Table 1.1 – Objectifs et Livrables du Projet

ID	Objectif	Livrable
O1	Construire un système d’authentification multi-rôles complet et sécurisé	API Auth, middleware JWT, OAuth, 2FA
O2	Activer les flux CRUD et de commande pour les appareils connectés	API Appareils, pipeline de publication MQTT
O3	Intégrer un courtier MQTT pour la messagerie IoT	Courtier Aedes, routage de topics, client ESP32
O4	Livrer la synchronisation d’état en temps réel aux clients web	Passerelle Socket.IO, catalogue d’événements
O5	Implémenter la surveillance d’urgence gaz avec réponse matérielle	Service Gas Monitor, workflow d’urgence
O6	Fournir un flux de demande de permission pour les résidents	API Permissions, système de notification
O7	Construire un plan d’étage 3D avec contrôle interactif des appareils	Composant FloorPlan, overlay SVG
O8	Développer un assistant IA (Melo) avec contrôle en langage naturel	API Companion, interface chat Melo
O9	Implémenter l’automatisation par scénarios temporels	API Scénarios, planificateur node-cron
O10	Créer un tableau de bord de surveillance de la consommation énergétique	API Énergie, visualisation re-charts
O11	Déployer le système complet avec runbook opérationnel documenté	Config déploiement, annexe run-book

1.5 Approche Méthodologique — Scrum

La mise en œuvre a suivi le cadre agile **Scrum**, choisi pour ses cycles de livraison itératifs, l’intégration continue du feedback et la gestion transparente du backlog. Les rôles Scrum ont été mappés au contexte académique :

- **Product Owner** — Encadrant académique (validation des exigences fonctionnelles).
- **Scrum Master** — Auteur (facilitation des cérémonies, suivi de la vélocité).

— **Équipe de développement** — Auteur (développement full-stack et firmware).

Le projet a été livré en **six sprints** de deux semaines chacun, comme résumé dans le Tableau 1.2.

Table 1.2 – Résumé des Sprints

Sprint	Thème	Livrables Principaux	SP
1	Authentification et Gestion des Utilisateurs	Flux d'inscription, JWT, OAuth, 2FA, vérification email	34
2	Gestion des Appareils et MQTT	CRUD Appareils, courtier MQTT, firmware ESP32	39
3	Plan d'Étage et Synchronisation TR	Plan 3D, scènes/ambiances, passerelle Socket.IO	37
4	Systèmes de Sécurité	Moniteur gaz, DHT11, workflow d'urgence, vanne	36
5	Assistant IA et Automatisation	Melo IA, scénarios, surveillance énergétique	34
6	Portail Agence et Audit	Hub conciergerie, journaux d'audit, permissions	37
Total			217

1.6 Contributions Scientifiques et Techniques

Ce projet contribue au domaine de l'IoT et du génie logiciel web en plusieurs dimensions :

- Une **architecture pratique** combinant APIs REST, courtage MQTT et WebSockets dans un service Node.js cohérent.
- Un **modèle de gouvernance des rôles** adaptable aux contextes résidentiels et de gestion de propriétés multi-unités.
- Une **boucle de contrôle sécuritaire** liant les seuils de télémétrie PPM aux commandes matérielles et aux notifications multi-canal.
- Une **interface en langage naturel** alimentée par LLM, contrainte par le contrôle d'accès basé sur les rôles.

- Un **plan d'artefacts Scrum** applicable aux projets IoT similaires en contexte académique.

1.7 Organisation du Rapport

Ce rapport est structuré pour guider le lecteur des aspects conceptuels aux détails d'implémentation et opérationnels :

Chapitres 1–3 : Contexte du projet, analyse des besoins, état de l'art et évaluation technologique.

Chapitre 4 : Architecture système et modélisation UML (déploiement, composants, cas d'utilisation, classes et diagrammes de séquence).

Chapitres 5–8 : Implémentation sprint par sprint couvrant la base de données, les APIs backend, l'interface frontend et l'intégration IoT.

Chapitres 9–11 : Évaluation sécuritaire, stratégie de test, pipeline de déploiement et gestion de projet.

Chapitre 12 : Conclusion, rétrospective et feuille de route future.

Annexes (13–16) : Référence API complète, inventaire frontend, runbook d'installation et matrice de traçabilité.

Analyse des Besoins et Backlog Produit

2.1 Parties Prenantes et Acteurs

La plateforme SmartHome sert quatre rôles utilisateur distincts, chacun avec des responsabilités uniques et des interactions système spécifiques.

Table 2.1 – Acteurs Système et Responsabilités

Acteur	Rôle Système	Responsabilités
Agence	<i>agency</i>	Gère plusieurs maisons ; accède au hub conciergerie ; visualise les messages inter-propriétés ; surveille le statut en ligne des admins via SSE.
Admin Maison	<i>admin</i>	Possède une maison (code unique) ; gère tous les appareils, utilisateurs, scénarios et seuils de sécurité ; approuve les demandes de permission.
Résident	<i>resident</i>	Vit dans une maison ; contrôle les appareils de sa chambre assignée ; soumet des demandes de permission ; utilise l'assistant IA dans les limites accordées.
SuperAdmin	<i>superadmin</i>	Propriétaire solo avec privilèges combinés admin+résident ; gère sa maison sans compte admin séparé.
Nœud ESP32	Matériel IoT	Publie la télémétrie des capteurs (gaz, DHT) et s'abonne aux topics de commande via MQTT.

2.2 Besoins Fonctionnels

2.2.1 Authentification et Gestion des Utilisateurs

Table 2.2 – Besoins Fonctionnels — Authentification

ID	Besoin	Priorité
BF-A1	Le système doit supporter l'inscription email/mot de passe pour les rôles Admin, Résident et Agence.	Indispensable
BF-A2	L'inscription Admin doit générer un code maison unique (1 lettre + 4 chiffres).	Indispensable
BF-A3	L'inscription Résident doit exiger un code maison valide pour rejoindre un foyer.	Indispensable
BF-A4	L'inscription Agence doit exiger un code d'accès secret.	Indispensable
BF-A5	Le système doit émettre des tokens JWT (expiration 30 jours) lors d'une connexion réussie.	Indispensable
BF-A6	Les mots de passe doivent être hachés avec bcryptjs (10 rounds de sel) avant stockage.	Indispensable
BF-A7	Le système doit supporter la connexion Google OAuth 2.0 via Passport.js.	Souhaitable
BF-A8	Le système doit supporter l'authentification à deux facteurs TOTP (speakeasy).	Souhaitable
BF-A9	Les nouveaux comptes doivent vérifier leur adresse email avant l'accès complet.	Indispensable
BF-A10	Le système doit supporter la réinitialisation de mot de passe par token email limité dans le temps.	Indispensable

2.2.2 Gestion des Appareils

Table 2.3 – Besoins Fonctionnels — Gestion des Appareils

ID	Besoin	Priorité
BF-D1	Les admins doivent créer, lire, mettre à jour et supprimer des appareils intelligents.	Indispensable
BF-D2	Chaque appareil doit avoir un topic MQTT unique pour le routage des commandes.	Indispensable

ID	Besoin	Priorité
BF-D3	Le système doit supporter au moins 20 types d'appareils (lumières, portes, fenêtres, etc.).	Indispensable
BF-D4	Les appareils stockent luminosité (0–100), ouverturePct (0–100), couleur et pièce.	Indispensable
BF-D5	Les commandes d'appareils sont publiées au courtier MQTT et routées vers l'ESP32.	Indispensable
BF-D6	Les changements d'état des appareils sont diffusés à tous les clients web via Socket.IO.	Indispensable
BF-D7	Les résidents contrôlent uniquement les appareils de leur pièce assignée ou ceux explicitement autorisés.	Indispensable

2.2.3 Plan d'Étage et Visualisation

Table 2.4 – Besoins Fonctionnels — Plan d'Étage

ID	Besoin	Priorité
BF-F1	L'interface doit afficher un plan 3D isométrique interactif avec 6 pièces nommées.	Indispensable
BF-F2	Cliquer sur une pièce ou un appareil ouvre un panneau de contrôle en ligne.	Indispensable
BF-F3	Les couleurs des pièces changent dynamiquement selon l'état d'éclairage.	Souhaitable
BF-F4	Le système doit supporter 4 préréglages de scènes : Matin, Dîner, Cinéma, Sommeil.	Indispensable
BF-F5	L'activation d'une scène règle la luminosité par pièce et contrôle les fenêtres/vanne gaz.	Indispensable

2.2.4 Systèmes de Sécurité

Table 2.5 – Besoins Fonctionnels — Sécurité

ID	Besoin	Priorité
BF-S1	L'ESP32 publie les lectures du capteur MQ6 toutes les 3 secondes via MQTT.	Indispensable
BF-S2	Le backend compare le PPM gaz à un seuil configurable (défaut : 400 PPM).	Indispensable

ID	Besoin	Priorité
BF-S3	Le dépassement du seuil active le mode urgence dans le système.	Indispensable
BF-S4	Le mode urgence déclenche : buzzer matériel, fermeture vanne gaz, alerte Socket.IO.	Indispensable
BF-S5	Les admins peuvent manuellement lever l'état d'urgence une fois le danger résolu.	Indispensable
BF-S6	L'ESP32 publie les lectures DHT11 de température et humidité toutes les 10 secondes.	Indispensable
BF-S7	Un overlay plein écran s'affiche dans l'interface pendant une urgence gaz.	Indispensable

2.2.5 Assistant IA, Scénarios et Énergie

Table 2.6 – Besoins Fonctionnels — IA, Scénarios et Énergie

ID	Besoin	Priorité
BF-IA1	Le système expose un assistant IA (Melo) alimenté par l'API Groq LLM.	Souhaitable
BF-IA2	Melo analyse les commandes en langage naturel et les traduit en actions sur les appareils.	Souhaitable
BF-IA3	Melo respecte le rôle et les permissions de l'utilisateur lors de l'exécution des commandes.	Indispensable
BF-SC1	Les admins définissent des scénarios d'automatisation temporels avec des heures de déclenchement HH :MM.	Souhaitable
BF-SC2	Un planificateur de fond exécute les scénarios à l'heure configurée (node-cron).	Souhaitable
BF-EN1	Le système estime la consommation énergétique horaire des appareils à partir des journaux d'audit.	Optionnel

2.3 Besoins Non Fonctionnels

Table 2.7 – Besoins Non Fonctionnels

ID	Catégorie	Besoin
BNF-P1	Performance	Le cycle commande (API → MQTT → ESP32 → maj état) doit s'effectuer en moins de 500 ms sur réseau local.
BNF-P2	Performance	L'API REST supporte 200 requêtes/15 min par IP sans dégradation.
BNF-S1	Sécurité	Les mots de passe sont stockés sous forme de hachages bcrypt ; le stockage en clair est interdit.
BNF-S2	Sécurité	Les routes protégées rejettent les requêtes non authentifiées avec HTTP 401.
BNF-S3	Sécurité	Helmet configure les en-têtes de sécurité (CSP, X-Frame-Options, HSTS).
BNF-S4	Sécurité	Les endpoints d'auth sont limités à 20 requêtes/15 min par IP.
BNF-R1	Fiabilité	Les moniteurs de capteurs de fond survivent aux erreurs d'analyse de messages MQTT individuels.
BNF-U1	Utilisabilité	L'interface est entièrement responsive sur écrans ≥ 768 px.
BNF-U2	Utilisabilité	Les thèmes sombre et clair sont disponibles et persistés dans localStorage.
BNF-SC1	Scalabilité	Le système supporte au moins 10 codes maison simultanés sans modification architecturale.

2.4 Backlog Produit

Le Backlog Produit est la liste priorisée de tous les éléments de travail. Les points de story ont été estimés en utilisant la séquence Fibonacci (1, 2, 3, 5, 8, 13).

Table 2.8 – Backlog Produit Complet

ID	User Story	Priorité	PS	Sprint
US-01	En tant qu'Admin, je veux m'inscrire avec email/mot de passe pour gérer ma maison.	Haute	5	1
US-02	En tant que Résident, je veux m'inscrire avec un code maison pour rejoindre un foyer.	Haute	3	1

ID	User Story	Priorité	PS	Sprint
US-03	En tant qu'utilisateur, je veux me connecter et recevoir un token JWT.	Haute	3	1
US-04	En tant qu'utilisateur, je veux me connecter avec Google OAuth pour un accès rapide.	Moyenne	5	1
US-05	En tant qu'utilisateur, je veux vérifier mon email avant d'utiliser la plateforme.	Haute	5	1
US-06	En tant qu'utilisateur, je veux réinitialiser mon mot de passe via un lien email sécurisé.	Haute	5	1
US-07	En tant qu'utilisateur, je veux activer la 2FA via une application d'authentification.	Moyenne	8	1
US-08	En tant qu'Admin, je veux voir et gérer tous les résidents de ma maison.	Haute	5	1
US-09	En tant qu'Admin, je veux ajouter, modifier et supprimer des appareils intelligents.	Haute	8	2
US-10	En tant qu'Admin, je veux contrôler les appareils (on/off/luminosité) depuis le tableau de bord.	Haute	8	2
US-11	En tant qu'utilisateur, je veux que les commandes d'appareils soient envoyées à l'ESP32 via MQTT.	Haute	8	2
US-12	En tant qu'utilisateur, je veux que l'ESP32 publie les lectures du capteur gaz toutes les 3 secondes.	Haute	5	2
US-13	En tant qu'utilisateur, je veux que l'ESP32 publie la température/humidité toutes les 10 secondes.	Haute	5	2
US-14	En tant qu'utilisateur, je veux voir un plan d'étage 3D isométrique interactif.	Haute	13	3
US-15	En tant qu'utilisateur, je veux cliquer sur une pièce pour contrôler ses appareils.	Haute	8	3
US-16	En tant qu'utilisateur, je veux activer des scènes d'éclairage (Matin/Dîner/Cinéma/Sommeil).	Haute	8	3
US-17	En tant qu'utilisateur, je veux que les changements d'état se reflètent en temps réel.	Haute	8	3
US-18	En tant qu'Admin, je veux une alerte plein écran lors de la détection d'une fuite gaz.	Haute	8	4
US-19	En tant que système, je veux fermer automatiquement la vanne et activer le buzzer.	Haute	8	4

ID	User Story	Priorité	PS	Sprint
US-20	En tant qu'Admin, je veux lever manuellement l'état d'urgence.	Haute	5	4
US-21	En tant qu'utilisateur, je veux voir la température et l'humidité en temps réel.	Moyenne	5	4
US-22	En tant qu'Admin, je veux configurer le seuil de détection gaz.	Moyenne	5	4
US-23	En tant qu'utilisateur, je veux un assistant IA (Melo) pour contrôler les appareils en langage naturel.	Moyenne	13	5
US-24	En tant qu'Admin, je veux créer des scénarios d'automatisation temporels.	Moyenne	8	5
US-25	En tant qu'Admin, je veux voir les graphiques de consommation énergétique horaire.	Moyenne	8	5
US-26	En tant qu'utilisateur, je veux voir l'historique de mes demandes de permission.	Moyenne	5	5
US-27	En tant qu'Agence, je veux un hub conciergerie avec le statut en direct des maisons.	Faible	8	6
US-28	En tant qu'Admin, je veux un journal d'audit filtrable de toutes les actions.	Moyenne	8	6
US-29	En tant que Résident, je veux soumettre des demandes de permission à l'admin.	Moyenne	5	6
US-30	En tant qu'Admin, je veux approuver ou refuser les demandes avec des notes.	Moyenne	5	6
US-31	En tant qu'Agence, je veux répondre aux messages des résidents sur toutes les maisons.	Faible	5	6
US-32	En tant qu'Admin, je veux configurer les modes de sécurité (Absence, Verrouillage).	Moyenne	5	6
US-33	En tant qu'utilisateur, je veux mettre à jour ma photo de profil et mes informations personnelles.	Faible	3	6
US-34	En tant qu'Admin, je veux exporter le journal d'audit en PDF.	Faible	3	6

Total points de story : 217 sur 6 sprints. Vitesse moyenne : 36 PS/sprint.

2.5 Diagramme de Cas d'Utilisation Global

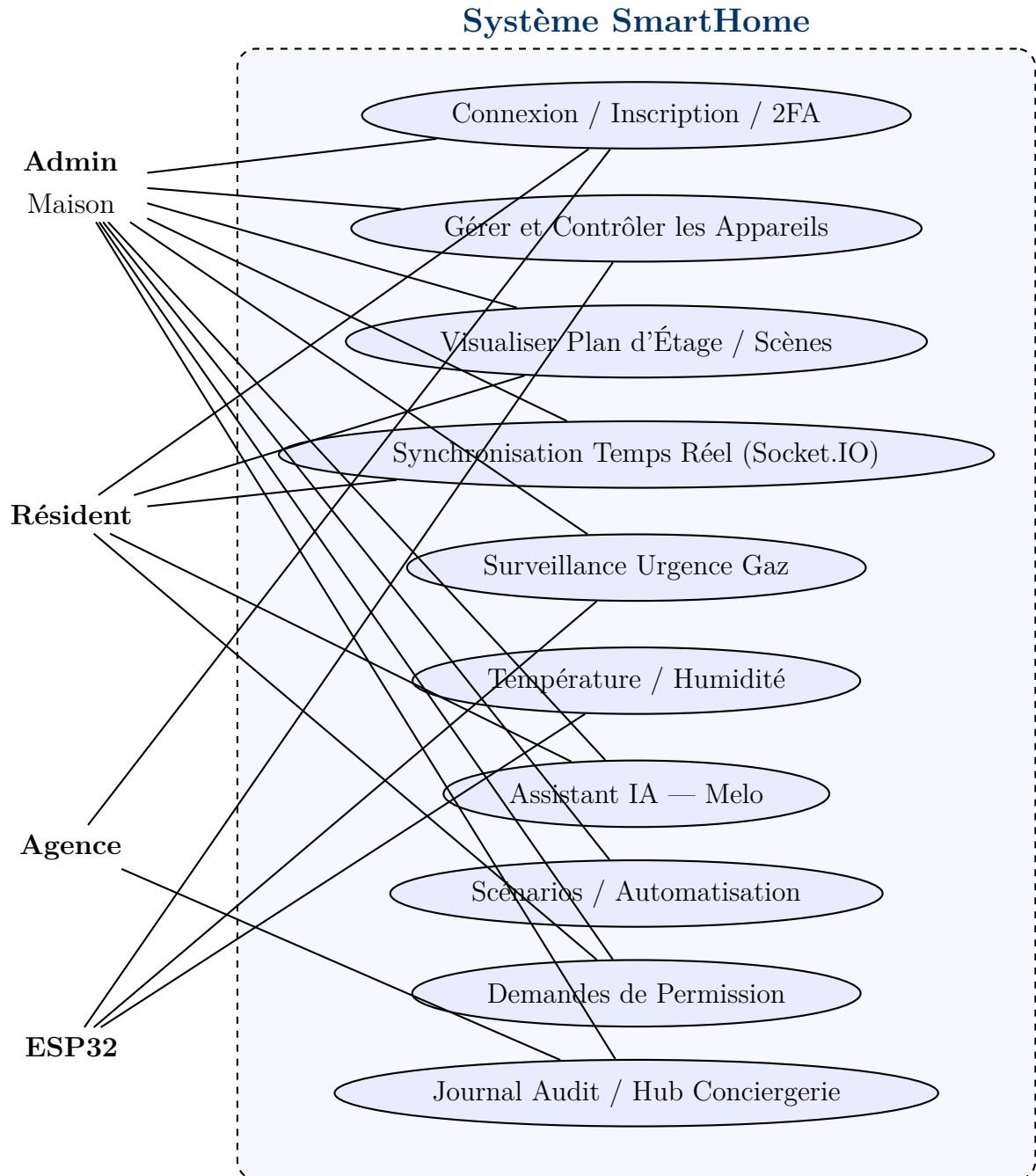


Figure 2.1 – Diagramme de Cas d'Utilisation Global de la Plateforme SmartHome

État de l'Art et Stack Technologique

3.1 Panorama des Plateformes de Maison Intelligente

Le marché de la maison intelligente est dominé par plusieurs écosystèmes propriétaires : **Amazon Alexa**, **Google Home**, **Apple HomeKit** et **Samsung SmartThings**. Malgré leurs richesses fonctionnelles, ces plateformes partagent des limitations communes :

- **Dépendance au fournisseur (vendor lock-in)** : les appareils doivent être certifiés pour l'écosystème spécifique.
- **Risques pour la vie privée** : les architectures cloud-first acheminent les commandes locales via des serveurs distants, créant de la latence et des risques d'exposition des données.
- **Personnalisation limitée** : la logique métier, la gouvernance des rôles et les flux de sécurité ne peuvent pas être étendus par l'utilisateur final.
- **Lacunes multi-locataires** : aucun modèle standard pour les scénarios de gestion d'agence ou de propriétés multi-unités.

Les alternatives open source comme **Home Assistant** et **openHAB** résolvent le problème de dépendance fournisseur, mais présentent des courbes de configuration abruptes et manquent de modèles de gouvernance multi-rôles intégrés.

Table 3.1 – Comparaison des Solutions de Maison Intelligente

Fonctionnalité	Amazon Alexa	Google Home	Home Asst.	openHAB	Ce Projet
Gouvernance multi-rôles	✗	✗	✗	✗	✓
Courtier MQTT intégré	✗	✗	✓	✓	✓
Sécurité gaz intégrée	✗	✗	Partiel	Partiel	✓
Assistant IA (LLM)	✓	✓	Partiel	✗	✓
Open source / auto-hébergé	✗	✗	✓	✓	✓
Mode agence/conciergerie	✗	✗	✗	✗	✓
Plan d'étage 3D	✗	✗	✓	✗	✓
Journal d'audit complet	✗	✗	Partiel	✗	✓

3.2 Protocoles de Communication IoT

3.2.1 MQTT (Message Queuing Telemetry Transport)

MQTT est un protocole de messagerie publish-subscribe léger conçu pour les appareils contraints et les réseaux peu fiables. Il fonctionne sur TCP et utilise un **courtier** comme routeur central de messages.

Table 3.2 – Caractéristiques Clés de MQTT

Caractéristique	Description
Overhead minimal	En-tête fixe aussi petit que 2 octets
Niveaux QoS	QoS 0 (au plus une fois), QoS 1 (au moins une fois), QoS 2 (exactement une fois)
Messages retenus	La dernière valeur publiée est livrée aux nouveaux abonnés
Last Will (LWT)	Le courtier publie un message prédéfini lors d'une déconnexion inattendue

Caractéristique	Description
Wildcards de topics	+ (niveau unique) et # (multi-niveau) pour les abonnements flexibles

Dans ce projet, **Aedes** est utilisé comme courtier MQTT en processus, intégré dans le backend Node.js, éliminant le besoin d'une instance Mosquitto ou HiveMQ externe.

3.2.2 Comparaison des Protocoles

Critère	MQTT	WebSocket	SSE HTTP	Polling REST
Adapté aux appareils IoT	✓	✗	✗	Partiel
Compatibilité navigateur	Partiel	✓	✓	✓
Bidirectionnel	✓	✓	✗	✗
Faible consommation	✓	Partiel	Partiel	✗
Push serveur	✓	✓	✓	✗

L'architecture choisie utilise **MQTT** pour l'IoT et **Socket.IO** pour les navigateurs, le backend servant de pont entre les deux protocoles.

3.3 Évaluation du Stack Technologique

3.3.1 Backend : Node.js et Express

Node.js avec son modèle de boucle d'événements est bien adapté aux applications à forte intensité d'I/O comme cette plateforme, où le serveur gère simultanément les requêtes HTTP, les événements MQTT, les connexions Socket.IO et les opérations de base de données. **Express.js** fournit une couche de routage minimale et non opinionnée.

3.3.2 Base de Données : MongoDB et Mongoose

MongoDB a été sélectionné pour sa flexibilité schématique lors des premières itérations de conception et son modèle de document JSON natif aligné avec JavaScript. **Mongoose** fournit la validation de schéma, les virtuels, les hooks et une API de requête cohérente.

3.3.3 Frontend : React et Vite

Table 3.3 – Bibliothèques Frontend Utilisées

Bibliothèque	Version	Rôle
React	19.2.4	Framework UI basé sur les composants
React Router	7.13.2	Routage côté client avec routes protégées
Tailwind CSS	4.2.2	CSS utilitaire pour le design responsive
Framer Motion	12.38.0	Animations et transitions déclaratives
Three.js	0.184.0	Rendu 3D WebGL pour le plan d'étage
React Three Fiber	9.x	Liaisons React pour Three.js
Recharts	3.8.1	Bibliothèque de graphiques SVG pour le tableau de bord énergétique
Socket.IO Client	4.8.3	Communication temps réel navigateur-serveur
mqtt.js	5.15.1	Client MQTT pour le navigateur
Axios	1.14.0	Client HTTP basé sur les promesses
jsPDF	4.2.1	Génération de PDF côté client (export audit)
Lucide React	1.7.0	Bibliothèque d'icônes SVG cohérente

3.3.4 Firmware IoT : Arduino / ESP32

Le microcontrôleur **ESP32-D** a été choisi pour :

- Dual-core Xtensa LX6 à 240 MHz avec 520 Ko SRAM.
- Wi-Fi intégré (802.11 b/g/n) et Bluetooth.
- Riche ensemble GPIO supportant ADC, PWM, UART, I2C et SPI.
- Vaste écosystème Arduino avec bibliothèques MQTT, JSON et servo.
- Logique 3,3 V compatible avec la plupart des capteurs.

3.3.5 Intégration IA : API Groq

Groq fournit une inférence LLM ultra-faible latence, atteignant un débit significativement plus rapide que les endpoints standard d'OpenAI. Pour un assistant IA interactif, le temps de réponse est critique pour l'expérience utilisateur.

Bibliothèque d'Auth	Version	Rôle
jsonwebtoken	9.0.2	Génération et vérification JWT
bcryptjs	2.4.3	Hachage des mots de passe
passport	0.7.x	Orchestration des middlewares d'auth
passport-google-oauth20	2.x	Stratégie Google OAuth 2.0
speakeasy	2.0.0	Génération et vérification TOTP/2FA
qrcode	1.5.4	Génération d'images QR code pour 2FA

3.4 Environnement de Développement

Outil	Rôle
VS Code + Claude Code CLI	IDE principal et assistance au codage par IA
Git + GitHub	Contrôle de version et collaboration
Postman	Tests API et documentation
Arduino IDE 2.x	Développement et flashage du firmware ESP32
MongoDB Compass	Inspection de base de données et développement de requêtes
MQTT Explorer	Surveillance des topics MQTT lors des tests d'intégration
Nodemon	Rechargement automatique du backend en développement
ESLint + Prettier	Qualité et formatage du code JavaScript

3.5 Synthèse

Le stack technologique sélectionné combine des **outils éprouvés par l'industrie** (Node.js, MongoDB, React) avec des **protocoles spécifiques IoT** (MQTT/Aedes, ESP32) et des **capacités IA émergentes** (Groq LLM), créant un système cohérent, maintenable et déployable. Les choix privilégient l'**ergonomie développeur**, la **performance runtime** et la **maturité de l'écosystème** pour un projet PFE à développeur unique livré en 12 semaines.

Architecture Système et Modélisation UML

4.1 Vision Architecturale

La Plateforme IoT Maison Intelligente suit une **architecture en couches événementielle** construite sur trois canaux de communication principaux :

1. **REST (HTTP/JSON)** — requête-réponse synchrone pour les opérations CRUD, l'authentification, la configuration et la récupération de données.
2. **MQTT (Publication-Abonnement)** — messagerie asynchrone à faible latence entre le courtier backend et les dispositifs IoT (ESP32).
3. **Socket.IO (WebSocket)** — canal bidirectionnel événementiel pour la diffusion d'état en temps réel aux clients navigateur.

Le backend joue le rôle de **hub d'intégration** : il héberge le courtier MQTT, l'API HTTP et la passerelle Socket.IO, traduisant les événements entre les trois canaux tout en persistant l'état dans MongoDB.

4.2 Diagramme de Déploiement

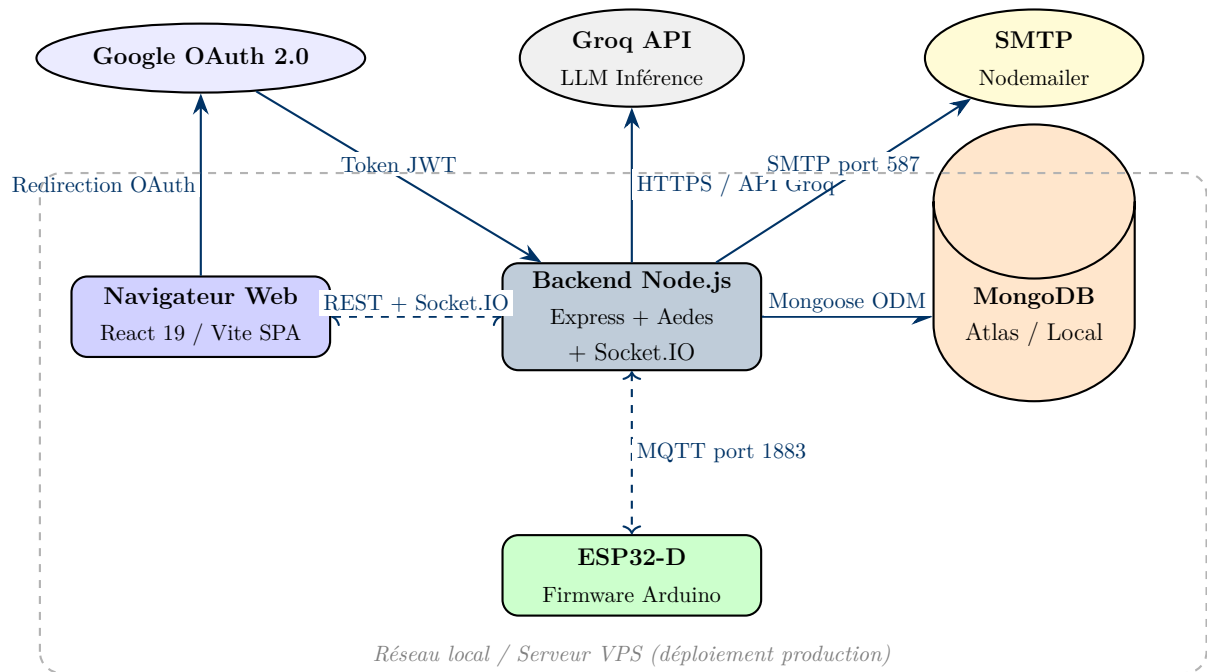


Figure 4.1 – Diagramme de Déploiement de la Plateforme SmartHome

4.3 Diagramme de Composants

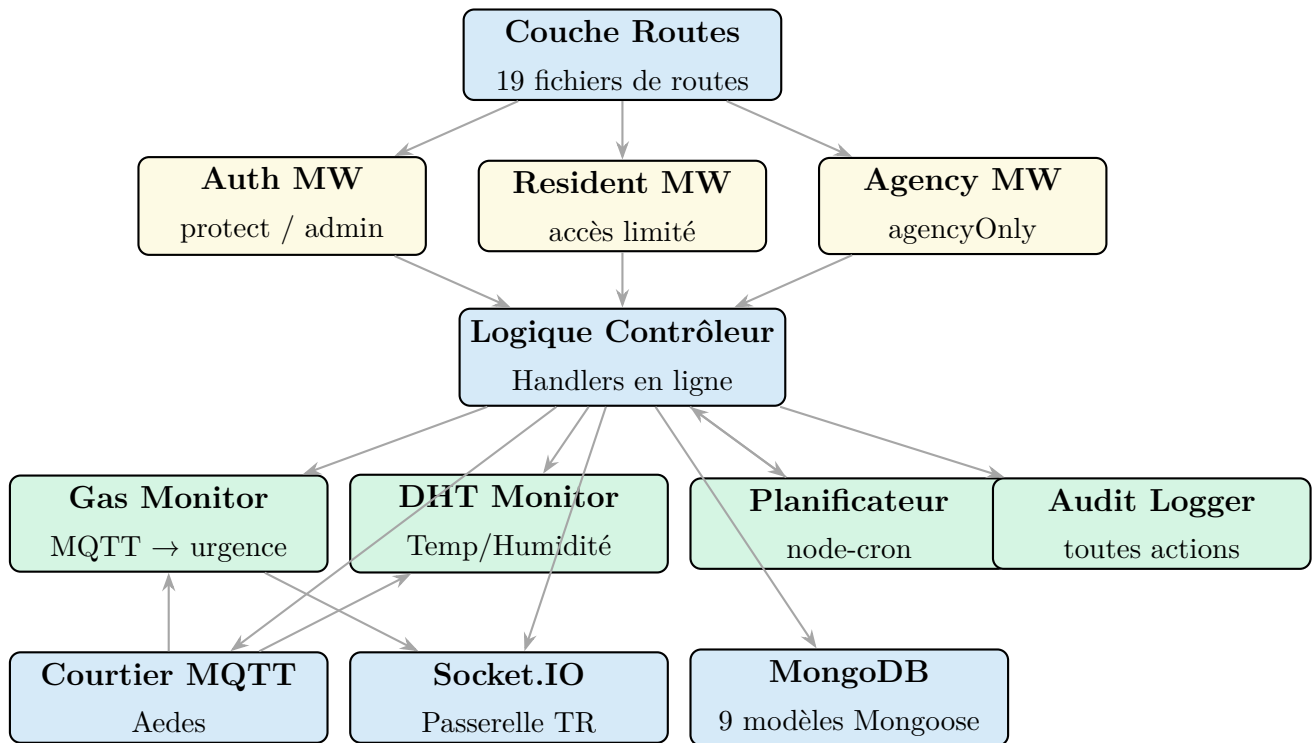


Figure 4.2 – Diagramme de Composants du Backend

4.4 Structure du Monorepo

Listing 4.1 – Structure du Répertoire du Projet

```

1 SmartHome-PFE/
2 |-- index.js # Point d'entree monorepo (démarré le backend)
3 |-- package.json # Scripts racine (concurrently dev)
4 |-- backend/
5 | |-- server.js # Application Express + configuration Socket.IO
6 | |-- broker.js # Configuration courtier MQTT Aedes
7 | |-- gasMonitor.js # Service telemetrie gaz + logique urgence
8 | |-- dhtMonitor.js # Service persistance telemetrie DHT11
9 | |-- scheduler.js # Exécuteur de scenarios node-cron
10 | |-- routes/ # 19 modules de routes
11 | |-- models/ # 9 schemas Mongoose
12 | |-- middlewares/ # protect, admin, resident, agency
13 | |-- realtime/ # Gestionnaires evenements Socket.IO
14 | |-- utils/ # mailer, auditLogger
15 | '-- .env # Secrets d'environnement
16 |-- frontend/

```

```

17 | |-- src/
18 | | |-- pages/ # 34 composants de pages
19 | | |-- components/ # 25+ composants réutilisables
20 | | |-- hooks/ # Hooks React personnalisés
21 | | |-- context/ # AuthContext, ThemeContext
22 | | |-- realtime/ # Intégration client Socket.IO
23 | | '-- App.jsx # Définitions de routes + gardes
24 | '-- vite.config.js
25 '-- esp32/
26     '-- SmartHome_ESP32/
27         '-- SmartHome_ESP32.ino # Firmware Arduino (~430 lignes)

```

4.5 Diagramme de Classes (Modèle Domaine)

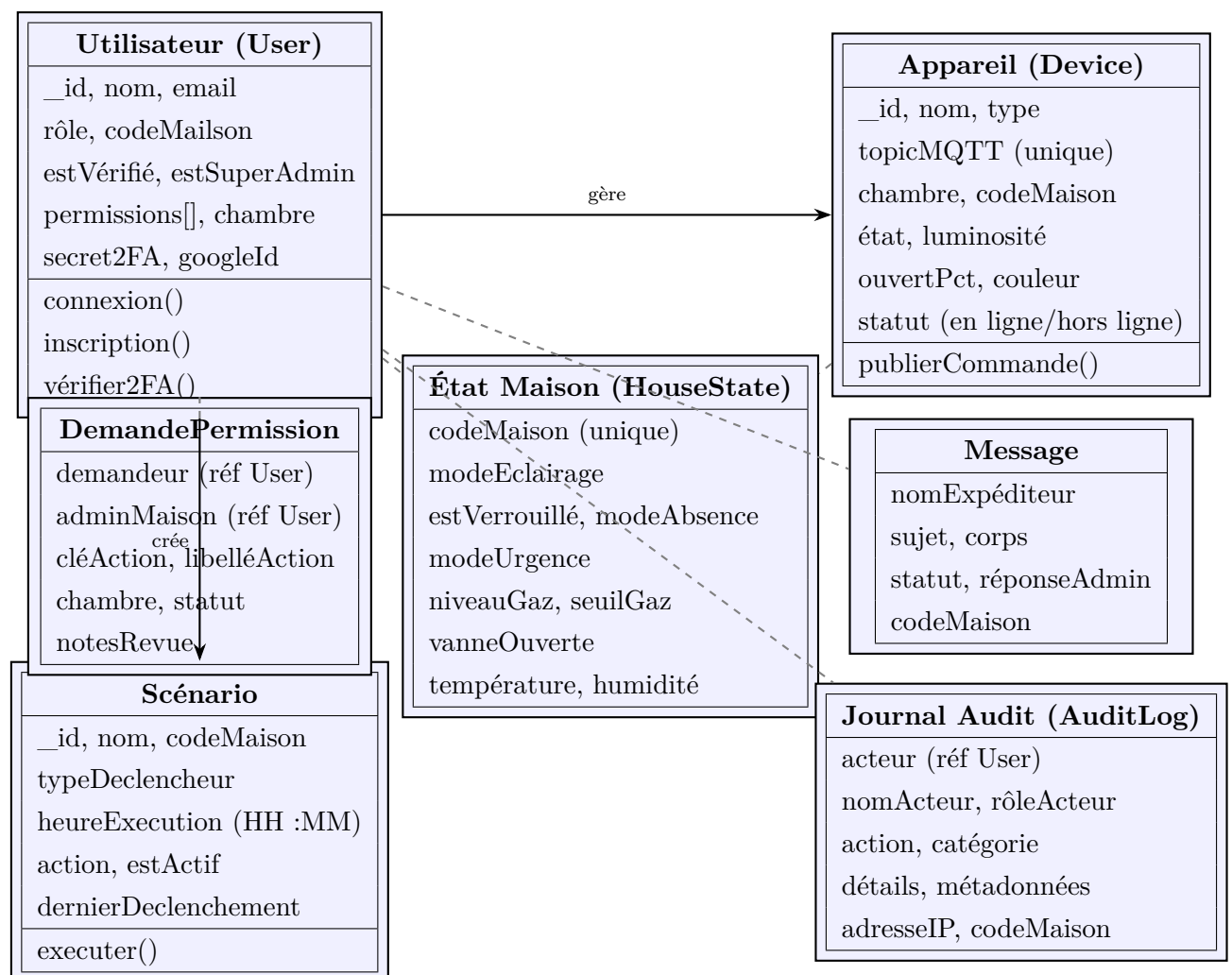


Figure 4.3 – Diagramme de Classes du Modèle Domaine

4.6 Diagramme de Séquence — Commande d'Appareil

Ce diagramme illustre le flux complet d'un clic utilisateur sur un bouton de contrôle jusqu'à la réponse physique de l'actionneur sur l'ESP32.

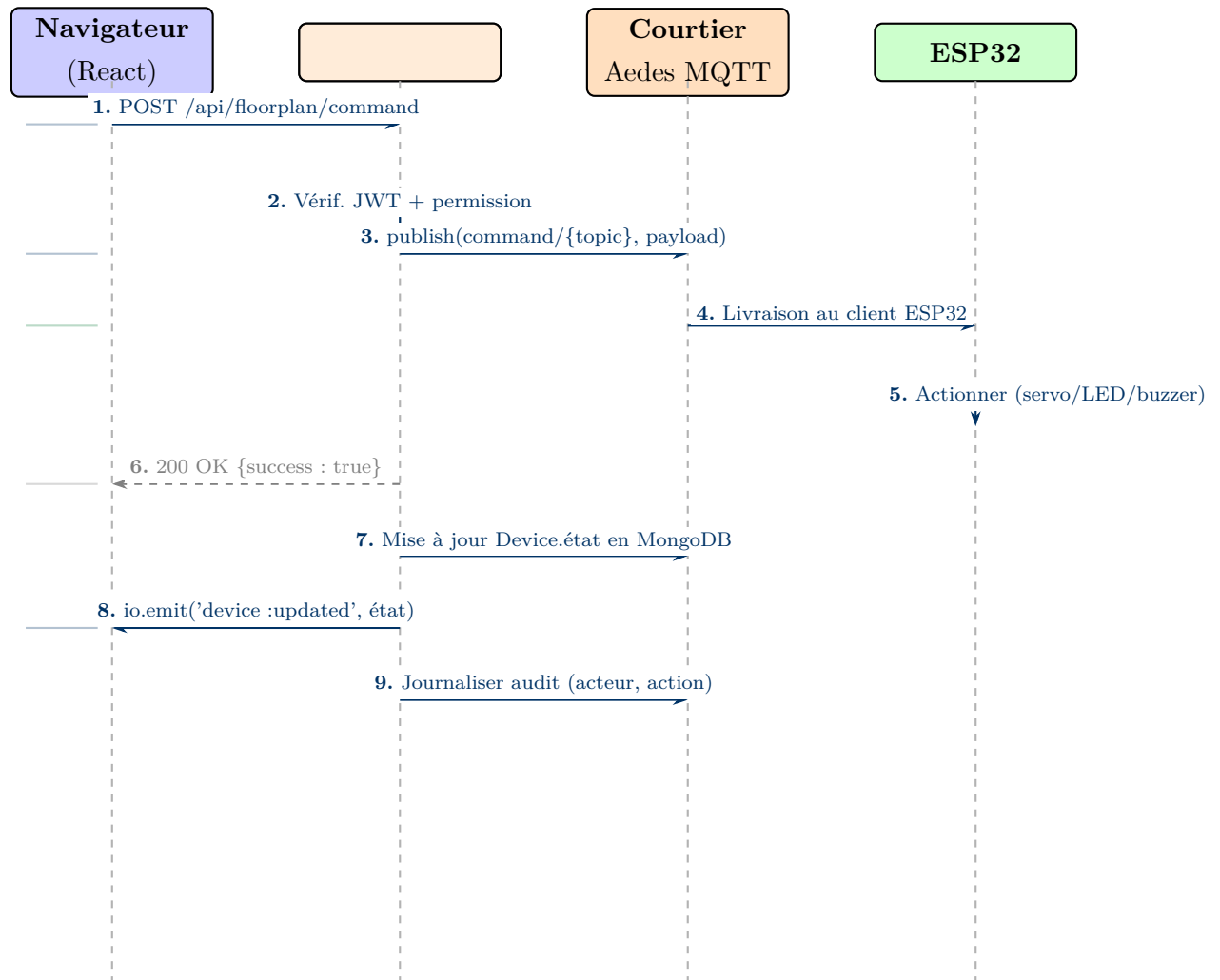


Figure 4.4 – Diagramme de Séquence — Flux de Commande d'Appareil

4.7 Diagramme de Séquence — Urgence Gaz

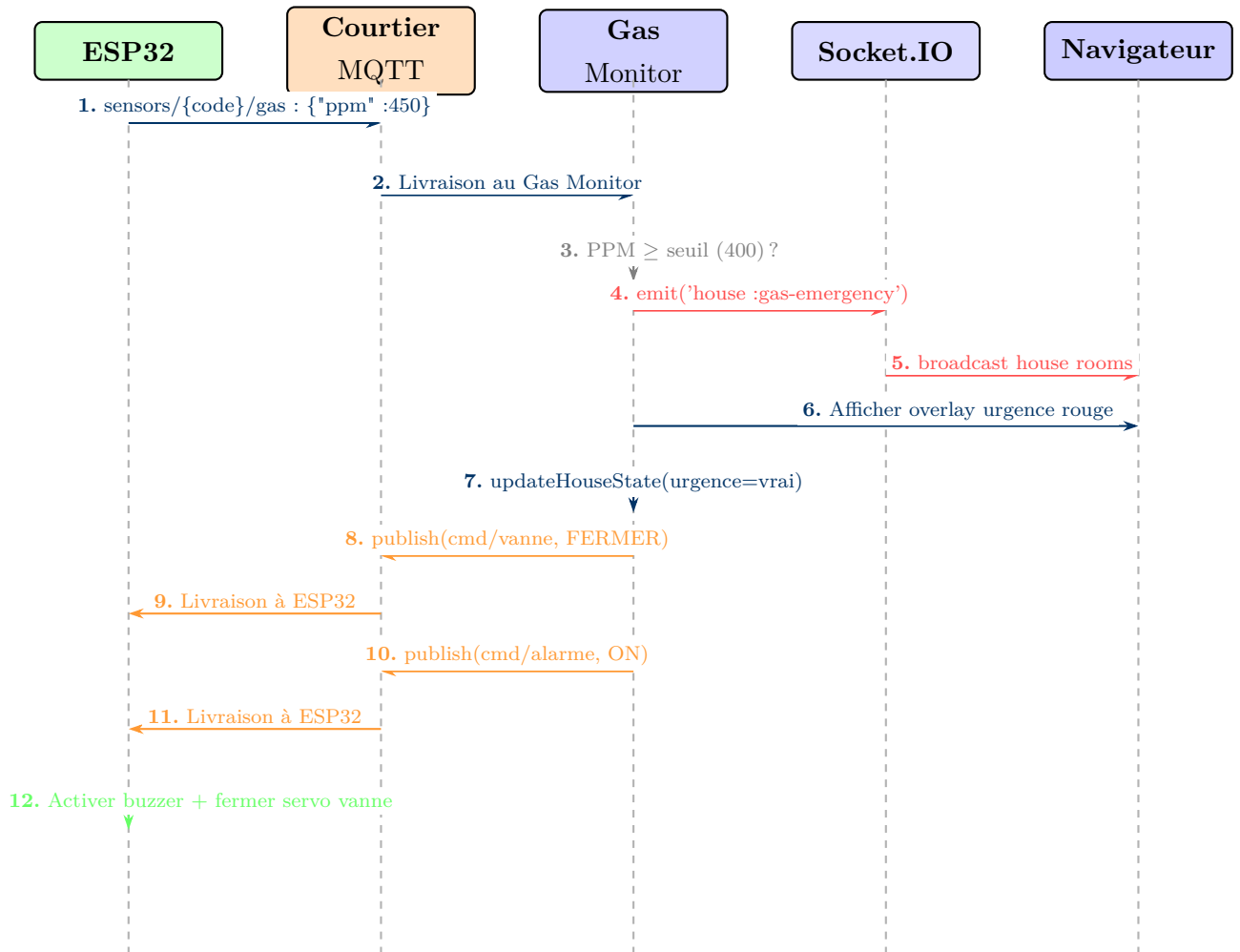


Figure 4.5 – Diagramme de Séquence — Détection et Réponse à l'Urgence Gaz

4.8 Machine à États — Mode Urgence

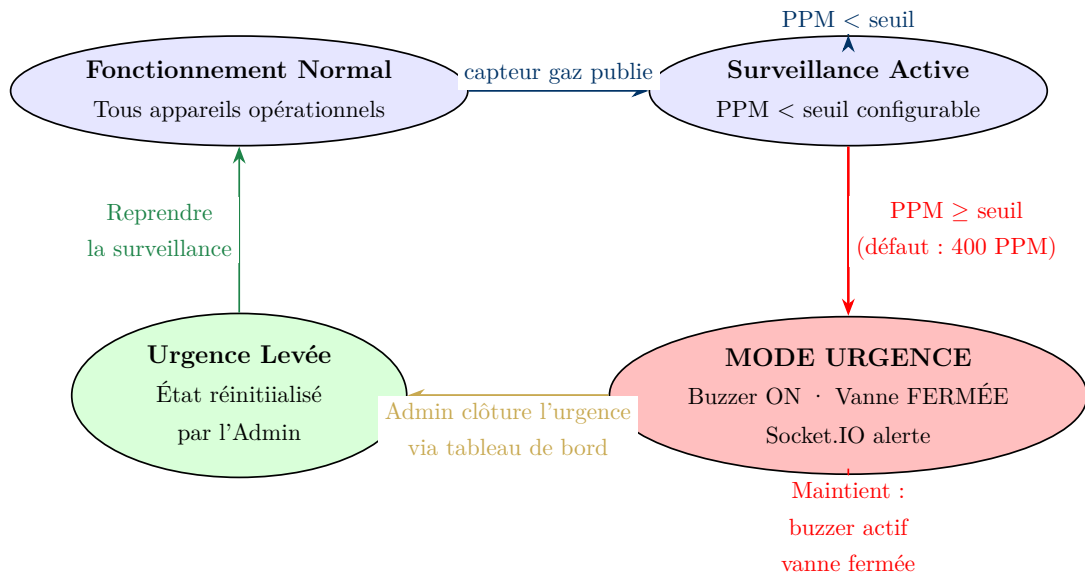


Figure 4.6 – Machine à États — Transitions du Mode Urgence Maison

4.9 Flux de Données Bout en Bout

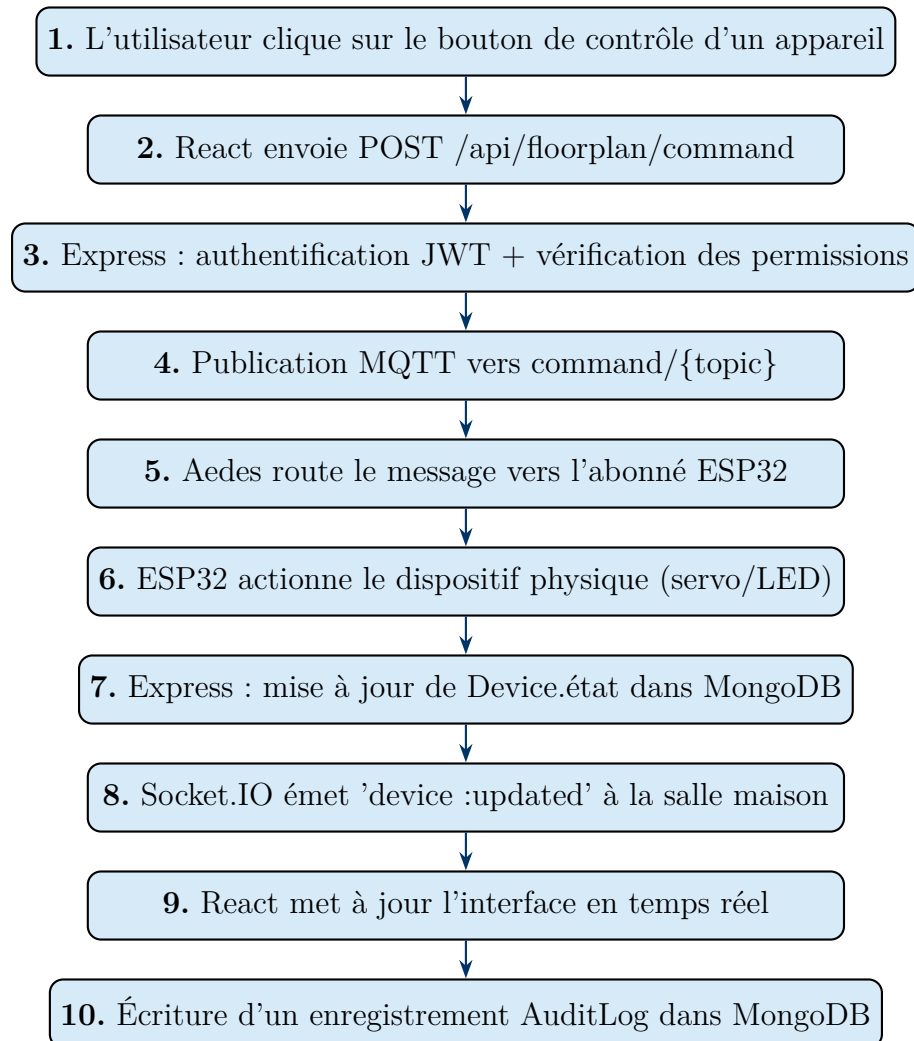


Figure 4.7 – Flux de Données de Commande d'Appareil Bout en Bout

Sprint 1 & 2 — Authentification, Gestion des Utilisateurs et Contrôle des Appareils

Sprint 1 — Authentification et Gestion des Utilisateurs

Durée : 2 semaines **Points de Story :** 34 **Objectif :** Livrer un système d'inscription et d'authentification multi-rôles complet et sécurisé, incluant la vérification email, Google OAuth et l'authentification à deux facteurs.

5.1 Backlog Sprint 1

Table 5.1 – Décomposition des User Stories — Sprint 1

ID	User Story	PS	Statut
US-01	Inscription Admin avec génération de code maison	5	✓Terminé
US-02	Inscription Résident avec validation du code maison	3	✓Terminé
US-03	Connexion email/mot de passe avec émission JWT	3	✓Terminé
US-04	Connexion Google OAuth 2.0	5	✓Terminé
US-05	Flux de vérification email	5	✓Terminé
US-06	Réinitialisation de mot de passe par token email	5	✓Terminé
US-07	Configuration de l'authentification à deux facteurs TOTP	8	✓Terminé

ID	User Story	PS	Statut
US-08	Gestion des utilisateurs Admin (liste/suppression résidents)	5	✓Terminé

5.2 Modèle de Base de Données : Schéma Utilisateur

La collection **User** est le magasin d'identité central. Elle supporte les quatre rôles et accumule toutes les informations d'authentification dans un seul document.

Listing 5.1 – Schéma Mongoose Utilisateur (backend/models/User.js)

```

1  const userSchema = new mongoose.Schema({
2    name: { type: String, required: true, trim: true },
3    email: { type: String, required: true, unique: true, lowercase: true },
4    password: { type: String, select: false }, // hachage bcrypt
5    role: { type: String,
6          enum: ['admin', 'resident', 'user', 'agency'],
7          default: 'user' },
8    houseCode: { type: String, default: null }, // 1 lettre + 4 chiffres
9    isVerified: { type: Boolean, default: false },
10   isSuperAdmin: { type: Boolean, default: false },
11   status: { type: String,
12          enum: ['active', 'pending', 'restricted'],
13          default: 'pending' },
14   permissions: [{ type: String }], // ex: 'room:cuisine'
15   room: { type: String, default: null },
16   googleId: { type: String, default: null },
17   verificationToken: { type: String, select: false },
18   verificationExpires: { type: Date, select: false },
19   resetToken: { type: String, select: false },
20   resetExpires: { type: Date, select: false },
21   twoFASecret: { type: String, select: false }, // secret TOTP base32
22   twoFAEnabled: { type: Boolean, default: false },
23   twoFATempSecret: { type: String, select: false },
24   avatar: { type: String, default: null }, // URI donnees base64
25 }, { timestamps: true });

```

5.2.1 Algorithme de Génération du Code Maison

Chaque admin se voit attribuer un code maison unique suivant le modèle [A-Z] [0-9] {4} (ex. : I5402). L'algorithme garantit l'unicité :

Listing 5.2 – Génération du Code Maison

```
1  const generateHouseCode = async () => {  
2    let code, exists;  
3    do {  
4      const letter = String.fromCharCode(65 + Math.floor(Math.random() * 26));  
5      const digits = Math.floor(1000 + Math.random() * 9000);  
6      code = `${letter}${digits}`;  
7      exists = await User.findOne({ houseCode: code });  
8    } while (exists);  
9    return code; // Unicité garantie en base de données  
10 };
```

5.3 Flux d'Authentification

5.3.1 Inscription et Connexion Email/Mot de Passe

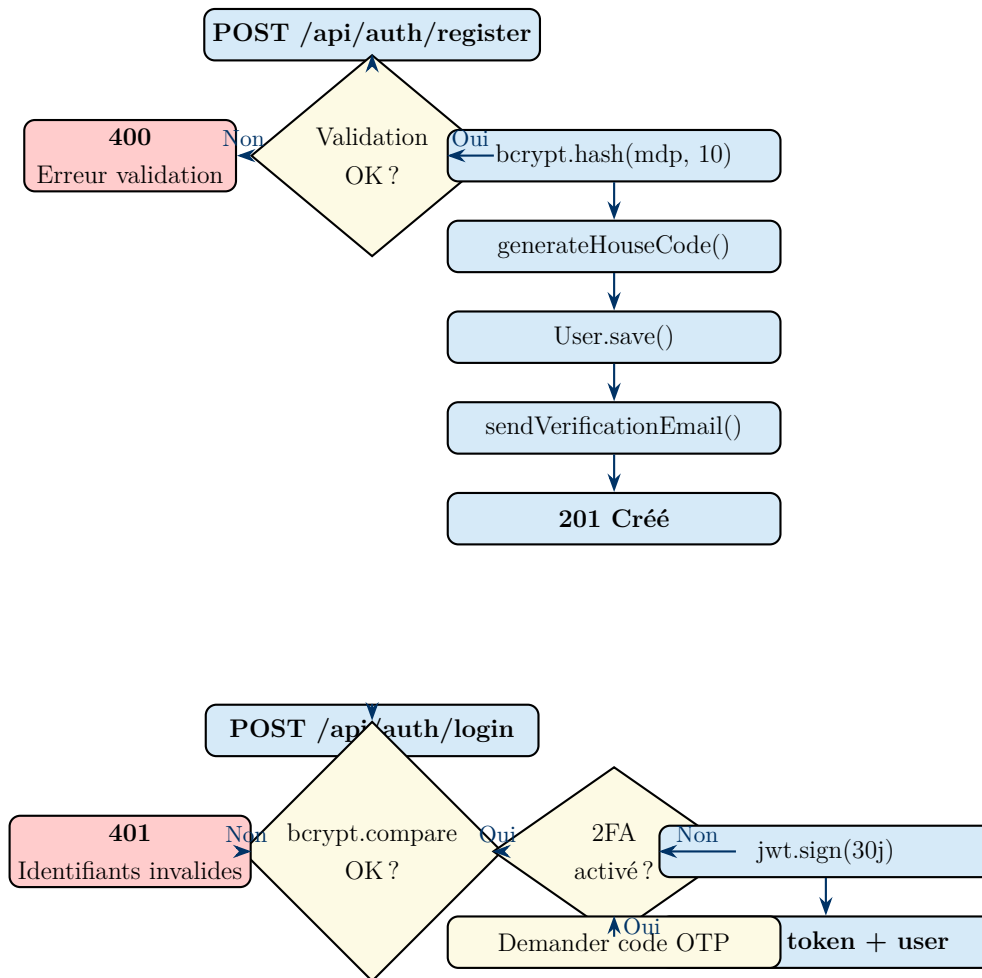


Figure 5.1 – Flux d’Inscription et de Connexion

5.3.2 Authentification à Deux Facteurs TOTP

Listing 5.3 – Configuration et Vérification 2FA (backend/routes/2fa.js)

```

1 // Etape 1 : Generer le secret TOTP et le QR code
2 router.post('/setup', protect, async (req, res) => {
3   const secret = speakeasy.generateSecret({
4     name: 'SmartHome:${req.user.email}'
5   });
6   req.user.twoFATempSecret = secret.base32;
7   await req.user.save();
8   const qrCode = await QRCode.toDataURL(secret.otppath_url);
9   res.json({ qrCode, secret: secret.base32 });
10 });
11
12 // Etape 2 : Verifier le code TOTP et activer la 2FA
13 router.post('/verify', protect, async (req, res) => {

```

```
14  const { token } = req.body;
15  const verified = speakeasy.totp.verify({
16    secret: req.user.twoFATempSecret,
17    encoding: 'base32',
18    token,
19    window: 1 // tolerance +/-30 secondes
20  });
21  if (!verified)
22    return res.status(400).json({ message: 'Code invalide' });
23
24  req.user.twoFASecret = req.user.twoFATempSecret;
25  req.user.twoFAEnabled = true;
26  await req.user.save();
27  res.json({ message: '2FA activee avec succes' });
28  });
```

5.4 Middleware JWT

Toutes les routes protégées utilisent le middleware `protect` pour valider le token JWT et attacher l'objet utilisateur à `req.user` :

Listing 5.4 – Middleware d'Authentification (backend/middlewares/authMiddleware.js)

```
1  const protect = async (req, res, next) => {
2    const header = req.headers.authorization;
3    if (!header?.startsWith('Bearer '))
4      return res.status(401).json({ message: 'Aucun token fourni' });
5
6    const token = header.split(' ')[1];
7    try {
8      const decoded = jwt.verify(token, process.env.JWT_SECRET);
9      req.user = await User.findById(decoded.id).select('+twoFASecret');
10     if (!req.user)
11       return res.status(401).json({ message: 'Utilisateur introuvable' });
12     next();
13   } catch {
14     res.status(401).json({ message: 'Token invalide ou expire' });
15   }
16  };
```

5.5 Revue et Rétrospective Sprint 1

Revue Sprint 1

Complété : 8 user stories sur 8 (34 PS).

Points forts de la démo : Inscription multi-rôles réussie, callback OAuth Google fonctionnel, scan QR code TOTP avec Google Authenticator, livraison d'email de réinitialisation.

Rétrospective :

- **Ce qui a bien fonctionné** : La conception du middleware JWT s'est révélée propre et réutilisable dans tous les 19 fichiers de routes.
- **Amélioration** : La vérification automatique des comptes Admin a été ajoutée en milieu de sprint pour éviter de bloquer le flux d'intégration pendant les tests.

Sprint 2 — Gestion des Appareils et Intégration MQTT

Durée : 2 semaines **Points de Story** : 39 **Objectif** : Implémenter le cycle de vie CRUD complet des appareils, intégrer le courtier MQTT Aedes, et déployer le premier flux de commande bout en bout vers l'ESP32.

5.6 Modèle Appareil

Listing 5.5 – Schéma Appareil (backend/models/Device.js)

```
1  const deviceSchema = new mongoose.Schema({
2    name: { type: String, required: true },
3    type: { type: String, required: true,
4      enum: ['light', 'lamp', 'door', 'lock', 'window', 'blind', 'ac',
5            'climate', 'thermostat', 'tv', 'camera', 'fan', 'outlet',
6            'washing-machine', 'gas-sensor', 'smoke', 'irrigation',
7            'rgb-light', 'valve', 'buzzer'] },
8    mqttTopic: { type: String, required: true, unique: true },
9    room: { type: String, required: true },
10   houseCode: { type: String, required: true },
```



```

11 owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
12 state: { type: mongoose.Schema.Types.Mixed, default: false },
13 brightness:{ type: Number, default: 100, min: 0, max: 100 },
14 openPct: { type: Number, default: 0, min: 0, max: 100 },
15 color: { type: String, default: '#FFFFFF' },
16 position: { x: { type: Number, default: 50 },
17             y: { type: Number, default: 50 } },
18 status: { type: String, enum: ['online','offline'], default: 'offline' },
19 lastSeen: { type: Date }
20 }, { timestamps: true });

```

5.6.1 Types d'Appareils et Puissance Estimée

Table 5.2 – Types d'Appareils et Consommation Estimée

Type d'Appareil	Mode de Contrôle	Puissance Est. (W)
light, lamp, rgb-light	ON/OFF + luminosité (0–100)	10
door, lock	OPEN/CLOSE (servo 0–180°)	5
window, blind	OPEN/CLOSE + ouverturePct	5
ac, climate	ON/OFF	1500
thermostat	ON/OFF	100
tv	ON/OFF	80
camera	ON/OFF	5
fan	ON/OFF	50
outlet	ON/OFF	200
washing-machine	ON/OFF	500
gas-sensor, smoke	Passif (lecture seule)	1
valve	OPEN/CLOSE	10
buzzer	ON/OFF	1

5.7 Cartographie GPIO de l'ESP32

Table 5.3 – Affectation des Broches GPIO ESP32

Composant	Broche(s)	Mode	Description
LED RGB 1 (Salon)	GPIO 2, 4, 5	Sortie/PWM	Canaux Rouge, Vert, Bleu
LED RGB 2 (Chambre)	GPIO 21, 22, 32	Sortie/PWM	Canaux Rouge, Vert, Bleu
LED RGB 3 (Cuisine)	GPIO 25, 26, 27	Sortie/PWM	Canaux Rouge, Vert, Bleu
Buzzer (alarme)	GPIO 15	Sortie	Tonalité 1000 Hz au déclenchement
Servomoteur (Porte)	GPIO 18	PWM 50 Hz	0°=Fermé, 180°=Ouvert
Servomoteur (Fenêtre)	GPIO 19	PWM 50 Hz	0°=Fermé, 180°=Ouvert
Capteur MQ6 (Gaz)	GPIO 34	Entrée ADC	Concentration gaz analogique
Capteur DHT11	GPIO 23	Numérique	Température + Humidité

5.8 Publication MQTT des Commandes

Les commandes sont publiées sur `command/{topicMQTT}` avec un payload spécifiant l'état cible. L'ESP32 s'abonne à `command/#` et analyse le suffixe du topic pour identifier quel actionneur contrôler.

Listing 5.6 – Callback MQTT ESP32 (SmartHome_ESP32.ino)

```

1 void mqttCallback(char* topic, byte* payload, unsigned int length) {
2   String topicStr(topic);
3   String msg;
4   for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
5   msg.trim();
6   msg.toUpperCase();
7
8   // Normalisation des variantes de payload
9   bool isOn = (msg=="ON" || msg=="TRUE" || msg=="OPEN" || msg=="UNLOCK" || msg=="1");
10  bool isOff = (msg=="OFF" || msg=="FALSE" || msg=="CLOSE" || msg=="LOCK" || msg=="0");
11
12  // Routage par topic
13  if (topicStr.endsWith("/alarm")) {
14    if (isOn) tone(BUZZER_PIN, 1000);
15    else noTone(BUZZER_PIN);

```

```
16     return;
17 }
18 // LEDs RGB
19 for (int i = 0; i < NUM_LEDS; i++) {
20     if (topicStr.endsWith(ledTopics[i])) {
21         int bri = isOn ? 100 : 0;
22         if (!isOn && !isOff) bri = msg.toInt();
23         setRGBBrightness(i, bri);
24         return;
25     }
26 }
27 // Servo porte
28 if (topicStr.endsWith("/door"))
29     { doorServo.write(isOn ? 180 : 0); return; }
30 // Servo fenetre
31 if (topicStr.endsWith("/window"))
32     { windowServo.write(isOn ? 180 : 0); return; }
33 }
```

5.9 Revue et Rétrospective Sprint 2

Revue Sprint 2

Complété : Tous les éléments du sprint (39 PS). Première réponse d'actionneur physique vérifiée.

Points forts : Cliquer sur un bouton de lumière dans le tableau de bord a allumé la LED RGB sur la maquette en moins de 200 ms.

Rétrospective :

- **Ce qui a bien fonctionné :** Aedes en processus a éliminé la surcharge de configuration.
- **Défi :** La logique des LEDs RGB à anode commune nécessite des valeurs PWM inversées (255 – valeur) — identifié lors de l'intégration matérielle.
- **Amélioration :** Ajout de qos:1 à toutes les publications de commandes pour garantir la livraison même lors de brèves reconnections Wi-Fi.

Sprint 3 — Visualisation du Plan d'Étage, Contrôle de Scènes et Synchronisation Temps Réel

Sprint 3 — Plan d'Étage et Synchronisation Temps Réel

Durée : 2 semaines **Points de Story :** 37 **Objectif :** Livrer un plan d'étage 3D isométrique interactif avec contrôle des appareils par pièce, quatre préréglages de scènes d'éclairage, et la synchronisation d'état en temps réel via Socket.IO.

6.1 Modèle État Maison (HouseState)

Listing 6.1 – Schéma HouseState (backend/models/HouseState.js)

```
1  const houseStateSchema = new mongoose.Schema({
2    houseCode: { type: String, required: true, unique: true },
3    lightingMode: { type: String,
4                  enum: ['Cinematic', 'Dinner', 'Morning', 'Sleep'],
5                  default: 'Morning' },
6    isLocked: { type: Boolean, default: false },
7    awayMode: { type: Boolean, default: false },
8    emergencyMode: { type: Boolean, default: false },
9    gas: {
10      level: { type: Number, default: 0 },
11      threshold: { type: Number, default: 400 },
12      valveOpen: { type: Boolean, default: true },
13      lastAlert: { type: Date }
14    },
15    temperature: { type: Number, default: null },
```

```

16   humidity: { type: Number, default: null },
17   updatedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
18 }, { timestamps: true });

```

6.2 Routes API du Plan d'Étage

Table 6.1 – Routes API du Plan d'Étage

Méthode	Endpoint	Description
GET	/api/floorplan/state	Retourner tous les appareils groupés par pièce + HouseState
POST	/api/floorplan/command	Exécuter une commande (publier MQTT + mettre à jour BD)
POST	/api/floorplan/scene	Activer un préréglage de scène d'éclairage
GET	/api/floorplan/house-state	Retourner le document HouseState courant
POST	/api/floorplan/sync-defaults	Réinitialiser les positions des appareils aux valeurs par défaut

6.3 Implémentation des Scènes d'Éclairage

Chaque préréglage de scène définit des valeurs de luminosité par pièce, l'état des fenêtres et l'état de la vanne gaz.

Listing 6.2 – Logique d'Activation de Scène (backend/routes/floorplan.js)

```

1  const SCENES = {
2    Morning: { brightness: { 'Living Room':100,'Kitchen':100,'Bedroom':100,
3                          'Bathroom':100,'Utility':100,'Garage':80 },
4              windowsOpen: true, valveOpen: true },
5    Dinner: { brightness: { 'Living Room':75,'Kitchen':75,'Bedroom':35,
6                          'Bathroom':50,'Utility':30,'Garage':40 },
7              windowsOpen: false, valveOpen: true },
8    Cinematic: { brightness: { 'Living Room':8,'Kitchen':12,'Bedroom':10,
9                          'Bathroom':15,'Utility':18,'Garage':20 },

```

```

10         windowsOpen: false, valveOpen: false },
11     Sleep: { brightness: { 'Living Room':15,'Kitchen':10,'Bedroom':3,
12                          'Bathroom':25,'Utility':10,'Garage':5 },
13             windowsOpen: false, valveOpen: false }
14 };
15
16 router.post('/scene', protect, async (req, res) => {
17     const { scene } = req.body;
18     const config = SCENES[scene];
19     if (!config) return res.status(400).json({ message: 'Scene inconnue' });
20
21     const devices = await Device.find({ houseCode: req.user.houseCode });
22     const updates = [];
23
24     for (const device of devices) {
25         if (['light','lamp','rgb-light'].includes(device.type)) {
26             const bri = config.brightness[device.room] ?? 50;
27             device.state = bri > 0;
28             device.brightness = bri;
29             updates.push(device.save());
30             mqttClient.publish('command/${device.mqttTopic}',
31                               JSON.stringify({ state: bri > 0 ? 'ON':'OFF', brightness: bri }));
32         }
33         if (['window','blind'].includes(device.type)) {
34             device.state = config.windowsOpen;
35             updates.push(device.save());
36             mqttClient.publish('command/${device.mqttTopic}',
37                               config.windowsOpen ? 'OPEN' : 'CLOSE');
38         }
39         if (device.type === 'valve') {
40             device.state = config.valveOpen;
41             updates.push(device.save());
42             mqttClient.publish('command/${device.mqttTopic}',
43                               config.valveOpen ? 'OPEN' : 'CLOSE');
44         }
45     }
46
47     await Promise.all(updates);
48     await HouseState.findOneAndUpdate(
49         { houseCode: req.user.houseCode },
50         { lightingMode: scene },
51         { upsert: true }
52     );
53
54     io.to(req.user.houseCode).emit('dashboard:state-updated', { scene });
55     await logAudit(req.user, 'Scene activee : ${scene}', 'device', req.ip);
56     res.json({ success: true, scene });

```

```
57 });
```

6.4 Passerelle Socket.IO Temps Réel

6.4.1 Authentification et Adhésion aux Chambres

Listing 6.3 – Authentification Socket.IO (backend/server.js)

```
1 io.use(async (socket, next) => {
2   const token = socket.handshake.auth.token;
3   if (!token) return next(new Error('Aucun token d\'authentification'));
4   try {
5     const decoded = jwt.verify(token, process.env.JWT_SECRET);
6     socket.user = await User.findById(decoded.id).lean();
7     next();
8   } catch {
9     next(new Error('Token invalide'));
10  }
11 });
12
13 io.on('connection', (socket) => {
14   const { houseCode, role, _id } = socket.user;
15   if (houseCode) socket.join(houseCode); // Chambre scopee a la maison
16   socket.join('role:${role}'); // Chambre de role
17   socket.join('user:${_id}'); // Chambre personnelle
18   if (role === 'admin') socket.join('house:admin');
19   if (role === 'agency') socket.join('agency:global');
20 });
```

6.4.2 Catalogue des Événements Socket.IO

Table 6.2 – Référence des Événements Socket.IO

Nom de l'Événement	Direction	Description du Payload
device:updated	Serveur → Client	Document appareil mis à jour
dashboard:state-updated	Serveur → Client	Synchronisation d'état en masse (scène, tous appareils)
house:gas-emergency	Serveur → Client	{ppm, seuil, codeMaison}

Nom de l'Événement	Direction	Description du Payload
house:gas-clear	Serveur → Client	{codeMaison}
house:dht-update	Serveur → Client	{température, humidité}
permission:created	Serveur → Client	Nouvelle demande de permission
permission:updated	Serveur → Client	Demande de permission mise à jour
admin:connected	Serveur → Agence	Admin passé en ligne
admin:disconnected	Serveur → Agence	Admin passé hors ligne

6.5 Plan d'Étage 3D Isométrique

Le plan d'étage est rendu comme une projection isométrique SVG avec des overlays interactifs. Six pièces sont définies avec des coordonnées de mise en page fixes :

Pièce	Position Grille	Appareils Hébergés
Salon	(0,0)	Lumières, TV, AC, Caméra
Cuisine	(1,0)	Lumières, Prises, Capteur Gaz, Vanne
Chambre	(0,1)	Lumières, AC, Fenêtre
Salle de Bain	(1,1)	Lumières, Ventilateur
Utilitaire	(2,0)	Lave-linge, Prises
Garage	(2,1)	Porte, Caméra, Lumières

Chaque tuile de pièce est un polygone parallélogramme rendu à un angle isométrique. La couleur de la pièce passe dynamiquement du gris foncé (lumières éteintes) au jaune chaud (lumières à pleine luminosité) en interpolant la couleur hexadécimale selon la luminosité moyenne des lumières actives dans cette pièce.

6.6 Revue et Rétrospective Sprint 3

Revue Sprint 3

Complété : 4 user stories sur 4 (37 PS). Plan d'étage interactif démontré.

Points forts : L'activation de la scène "Cinéma" a atténué toutes les lumières à 8–18% et le changement s'est reflété en temps réel dans un second onglet navigateur en 150 ms.

Rétrospective :

- **Ce qui a bien fonctionné :** Le scopage des chambres Socket.IO par code maison a élégamment géré l'isolation multi-locataire.
- **Défi :** La projection isométrique SVG a nécessité des calculs mathématiques importants pour les coordonnées de parallélogramme et le positionnement des overlays.
- **Amélioration :** Ajout de mises à jour UI optimistes pour éliminer la latence perçue sur les réseaux locaux lents.

Sprint 4 — Détection Gaz, Capteur DHT11 et Protocoles d'Urgence

Sprint 4 — Systèmes de Sécurité

Durée : 2 semaines **Points de Story :** 36 **Objectif :** Implémenter la détection d'urgence gaz de bout en bout depuis le capteur MQ6 jusqu'à l'actuation matérielle et l'alerte UI plein écran, plus la surveillance de température et humidité DHT11.

7.1 Service de Fond Gas Monitor

Le service `gasMonitor.js` est le cerveau sécuritaire de la plateforme. Il s'abonne à tous les topics de capteurs gaz, évalue les lectures PPM par rapport au seuil configuré, et pilote la machine à états d'urgence.

Listing 7.1 – Service Gas Monitor (backend/gasMonitor.js)

```
1  const initGasMonitor = (mqttClient, io) => {
2    mqttClient.subscribe('sensors/+/gas');
3
4    mqttClient.on('message', async (topic, message) => {
5      const parts = topic.split('/');
6      if (parts.length !== 3 || parts[2] !== 'gas') return;
7      const houseCode = parts[1];
8
9      let ppm;
10     try {
11       ppm = JSON.parse(message.toString()).ppm;
12     } catch { return; } // Ignorer les messages malformés
13   }
```

```

14   let state = await HouseState.findOne({ houseCode });
15   if (!state) state = await HouseState.create({ houseCode });
16
17   const threshold = state.gas.threshold ?? 400;
18   state.gas.level = ppm;
19   await state.save();
20
21   if (ppm >= threshold && !state.emergencyMode) {
22     // ---- ACTIVATION DE L'URGENCE ----
23     state.emergencyMode = true;
24     state.gas.lastAlert = new Date();
25     state.gas.valveOpen = false;
26     await state.save();
27
28     // 1. Alerter tous les clients web de la maison
29     io.to(houseCode).emit('house:gas-emergency', { ppm, threshold, houseCode });
30
31     // 2. Fermer la vanne gaz
32     mqttClient.publish('command/${houseCode}/valve', 'CLOSE', { qos: 1 });
33
34     // 3. Activer le buzzer
35     mqttClient.publish('command/${houseCode}/alarm', 'ON', { qos: 1 });
36
37     // 4. Journaliser l'audit
38     await AuditLog.create({
39       actorName: 'Systeme',
40       actorRole: 'system',
41       action: 'Urgence gaz : ${ppm.toFixed(0)} PPM depasse le seuil ${threshold}',
42       category: 'security',
43       houseCode
44     });
45   }
46   });
47   };

```

7.2 Calcul PPM du Capteur Gaz MQ6

Le capteur MQ6 produit une tension analogique proportionnelle à la concentration de gaz. L'ESP32 lit cette valeur via l'ADC et la convertit en PPM en utilisant une formule calibrée :

Listing 7.2 – Calcul PPM Capteur Gaz (ESP32)

```

1 float readGasPPM() {
2     int raw = analogRead(GAS_PIN); // 0 - 4095 (ADC 12 bits)
3     float voltage= raw * 3.3f / 4095.f;
4     float Rs = ((3.3f - voltage) / voltage) * 10.0f; // RL = 10 kOhm
5     float ratio = Rs / 9.83f; // calibration Ro
6     return 1000.0f * pow(ratio, -2.37f); // PPM approximation
7 }
8
9 // Publier toutes les 3 secondes
10 if (millis() - lastGasPublish >= 3000) {
11     float ppm = readGasPPM();
12     String payload = "{\"ppm\":\"" + String(ppm, 1) + "\"}";
13     mqttClient.publish(gasTopic.c_str(), payload.c_str());
14
15     // Securite locale : declencher le buzzer si ADC brut > 800
16     if (analogRead(GAS_PIN) > 800) tone(BUZZER_PIN, 1000);
17     else noTone(BUZZER_PIN);
18     lastGasPublish = millis();
19 }

```

7.3 Service DHT11 et Firmware

Listing 7.3 – Lecture DHT11 et Publication MQTT (ESP32)

```

1 #include <DHT.h>
2 DHT dht(DHT_PIN, DHT11);
3
4 // Dans loop() :
5 if (millis() - lastDHTPublish >= 10000) {
6     float temperature = dht.readTemperature();
7     float humidity = dht.readHumidity();
8
9     if (!isnan(temperature) && !isnan(humidity)) {
10         String payload = "{\"temperature\":\"" + String(temperature, 1)
11             + "\", \"humidity\":\"" + String(humidity, 1) + "\"}";
12         mqttClient.publish(dhtTopic.c_str(), payload.c_str());
13     }
14     lastDHTPublish = millis();
15 }

```

7.4 Overlay d'Urgence Frontend

Listing 7.4 – Composant GasEmergencyOverlay

```

1  const GasEmergencyOverlay = ({ ppm, onClear, isAdmin }) => (
2    <motion.div
3      className="fixed inset-0 z-[9999] bg-red-600/90 flex flex-col
4        items-center justify-center text-white"
5      animate={{ opacity: [1, 0.8, 1] }}
6      transition={{ repeat: Infinity, duration: 1.2 }}
7    >
8      <AlertTriangle size={96} className="mb-6 animate-bounce" />
9      <h1 className="text-5xl font-bold mb-4">FUIITE DE GAZ DETECTEE</h1>
10     <p className="text-2xl mb-2">Niveau actuel : {ppm?.toFixed(0)} PPM</p>
11     <p className="text-lg mb-8 opacity-80">
12       Vanne fermee - Alarme active - Evacuez immediatement
13     </p>
14     {isAdmin && (
15       <button onClick={onClear}
16         className="px-8 py-3 bg-white text-red-700 rounded-full
17           font-bold text-lg hover:bg-red-100 transition">
18         Lever l'Urgence
19       </button>
20     )}
21   </motion.div>
22 );

```

7.5 Revue et Rétrospective Sprint 4

Revue Sprint 4

Complété : 6 éléments sur 6 (36 PS). Urgence gaz vérifiée sur le matériel.

Points forts : Souffler sur le capteur MQ6 a déclenché l'overlay d'urgence en 500 ms, activé le buzzer physique, et le servo de vanne s'est fermé. L'admin a levé l'urgence depuis le tableau de bord et le buzzer s'est tu en 1 seconde.

Rétrospective :

- **Ce qui a bien fonctionné** : L'architecture de sécurité à deux couches (déclencheur local ESP32 ADC > 800 + seuil serveur 400 PPM) fournit une

défense en profondeur.

- **Défi** : Le capteur DHT11 retourne occasionnellement NaN lors des lectures à froid ; géré par un guard `isnan()` avant la publication MQTT.
- **Amélioration** : Ajout de `qos:1` aux commandes alarme et vanne pour survivre aux déconnexions MQTT brèves lors des scénarios d'urgence.

Sprint 5 — Compagnon IA (Melo), Automatisation par Scénarios et Sur- veillance Énergétique

Sprint 5 — Compagnon IA & Automatisation

Durée : 2 semaines **Points de Story :** 34 **Objectif :** Livrer le compagnon IA Melo pour le contrôle des appareils en langage naturel, l'automatisation temporelle via node-cron, et un tableau de bord de surveillance de la consommation énergétique.

8.1 Backlog Sprint 5

Table 8.1 – Décomposition des User Stories Sprint 5

ID	User Story	PS	Statut
US-23	Compagnon IA (Melo) pour contrôle d'appareils en langage naturel	13	✓Terminé
US-24	Scénarios d'automatisation temporels avec node-cron	8	✓Terminé
US-25	Graphiques de consommation énergétique horaire	8	✓Terminé
US-26	Historique et statut des demandes de permission	5	✓Terminé

8.2 Compagnon IA Melo

8.2.1 Architecture

Melo est construit sur l'**API Groq**, un service d'inférence cloud offrant des temps de réponse LLM inférieurs à la seconde. Le backend expose un seul endpoint `POST /api/companion/chat` qui :

1. Enrichit le message utilisateur avec un prompt système structuré contenant la liste des appareils courants, les lectures des capteurs et les permissions utilisateur.
2. Envoie l'historique de conversation à l'API Groq avec le modèle `llama-3.3-70b-versatile`.
3. Analyse la réponse LLM pour détecter les **blocs d'action** (commandes JSON structurées).
4. Exécute les actions validées via le même pipeline de commande d'appareils que l'API REST.
5. Retourne la réponse LLM et les résultats des actions au frontend.

8.2.2 Construction du Prompt Système

Listing 8.1 – Construction du Prompt Système Melo (backend/routes/companion.js)

```
1  const buildSystemPrompt = (user, devices, houseState) => {
2    const accessibleDevices = user.role === 'admin'
3      ? devices
4      : devices.filter(d =>
5        d.room === user.room ||
6        user.permissions.includes('room:${d.room.toLowerCase().replace(' ', '-')}')
7        ↪ ) ||
8        user.permissions.includes('global:controls')
9      );
10   return `
11   Tu es Melo, un assistant maison intelligente intelligent et amical.
12   Tu aides les utilisateurs a controler leurs appareils en langage naturel.
13
14   UTILISATEUR COURANT :
15   - Nom : ${user.name}
16   - Role : ${user.role}
17   - Piece assignee : ${user.room || 'Toutes les pieces (admin)'}
```



```

18
19 APPAREILS ACCESSIBLES (${accessibleDevices.length} au total) :
20 ${accessibleDevices.map(d =>
21   '- ${d.name} (${d.type}) dans ${d.room} | topic: ${d.mqttTopic} | etat: ${d.
      ↪ state}'
22   ).join('\n')}
23
24 ENVIRONNEMENT :
25 - Niveau gaz : ${houseState?.gas?.level?.toFixed(0) ?? 'N/A'} PPM
26 - Temperature : ${houseState?.temperature ?? 'N/A'}C
27 - Humidite : ${houseState?.humidity ?? 'N/A'}%
28 - Scene courante : ${houseState?.lightingMode ?? 'Aucune'}
29
30 CAPACITES :
31 - Pour controler un appareil, reponds avec un bloc ACTION :
32   [ACTION: {"deviceId":"<id>","command":{"state":"ON"|"OFF","brightness":0-100}}]
33 - Pour activer une scene : [SCENE: "Morning"|"Dinner"|"Cinematic"|"Sleep"]
34 - Pour des requetes d'information, reponds de maniere conversationnelle.
35 - Ne controle JAMAIS des appareils hors de la liste accessible.
36 - Si tu ne peux pas faire quelque chose (permission refusee), explique poliment.
37   '.trim();
38   };

```

8.2.3 Implémentation de l'Endpoint Chat

Listing 8.2 – Endpoint Chat Compagnon (backend/routes/companion.js)

```

1 router.post('/chat', protect, async (req, res) => {
2   const { messages } = req.body; // Historique de conversation du frontend
3
4   const devices = await Device.find({ houseCode: req.user.houseCode });
5   const houseState = await HouseState.findOne({ houseCode: req.user.houseCode });
6   const sysPrompt = buildSystemPrompt(req.user, devices, houseState);
7
8   // Appel API Groq
9   const completion = await groq.chat.completions.create({
10     model: 'llama-3.3-70b-versatile',
11     messages: [
12       { role: 'system', content: sysPrompt },
13       ...messages
14     ],
15     temperature: 0.3, // Basse temperature pour des commandes deterministes
16     max_tokens: 1024
17   });
18
19   const reply = completion.choices[0].message.content;
20

```

```

21 // Analyser et exécuter les blocs ACTION
22 const actionMatches = [...reply.matchAll(/\[ACTION: ({.*?})\]/g)];
23 const results = [];
24 for (const match of actionMatches) {
25   try {
26     const { deviceId, command } = JSON.parse(match[1]);
27     const device = devices.find(d => d._id.toString() === deviceId);
28     if (device) {
29       await executeDeviceCommand(device, command, mqttClient, io, req.user);
30       results.push({ deviceId, success: true });
31     }
32   } catch { /* Ignorer les blocs d'action malformés */ }
33 }
34
35 // Analyser les blocs SCENE
36 const sceneMatch = reply.match(/\[SCENE: "([^\"]+)"\]/);
37 if (sceneMatch) {
38   await activateScene(sceneMatch[1], req.user.houseCode, mqttClient, io);
39 }
40
41 res.json({ reply, actionsExecuted: results.length });
42 });

```

8.2.4 Composant Frontend Melo

L'interface Melo est un widget de chat flottant rendu dans le coin inférieur droit de l'application. Elle inclut une mascotte 3D animée (Melo3D) construite avec React Three Fiber et animée avec Framer Motion.

Listing 8.3 – Interface Chat AuraCompanion (frontend)

```

1 const AuraCompanion = () => {
2   const [open, setOpen] = useState(false);
3   const [messages, setMessages] = useState([]);
4   const [input, setInput] = useState('');
5   const [loading, setLoading] = useState(false);
6   const { token } = useAuth();
7
8   const sendMessage = async () => {
9     const userMsg = { role: 'user', content: input };
10    setMessages(prev => [...prev, userMsg]);
11    setInput('');
12    setLoading(true);
13    try {
14      const { data } = await axios.post('/api/companion/chat',
15        { messages: [...messages, userMsg] },
16        { headers: { Authorization: `Bearer ${token}` } });

```

```

17     setMessages(prev => [
18       ...prev, { role: 'assistant', content: data.reply }
19     ]);
20   } finally {
21     setLoading(false);
22   }
23 };
24
25 return (
26   <div className="fixed bottom-6 right-6 z-50">
27     <AnimatePresence>
28       {open && (
29         <motion.div
30           initial={{ scale: 0, opacity: 0 }}
31           animate={{ scale: 1, opacity: 1 }}
32           exit={{ scale: 0, opacity: 0 }}
33           className="w-80 h-[450px] bg-white dark:bg-gray-900
34             rounded-2xl shadow-2xl flex flex-col overflow-hidden"
35         >
36           <div className="bg-indigo-600 p-3 text-white font-bold
37             flex items-center gap-2">
38             <Sparkles size={18} /> Melo -- Votre IA Maison Intelligente
39           </div>
40           <div className="flex-1 overflow-y-auto p-3 space-y-3">
41             {messages.map((m, i) => (
42               <div key={i} className={`flex
43                 ${m.role === 'user' ? 'justify-end' : 'justify-start'}>
44                 <div className={`rounded-xl px-3 py-2 max-w-[80%] text-sm
45                   ${m.role === 'user'
46                     ? 'bg-indigo-100'
47                     : 'bg-gray-100 dark:bg-gray-800'}>
48                   {m.content
49                     .replace(/\[ACTION:.*?\]/g, '')
50                     .replace(/\[SCENE:.*?\]/g, '')}
51                 </div>
52               </div>
53             ))}
54             {loading && (
55               <div className="text-xs text-gray-400 animate-pulse">
56                 Melo réfléchit...
57               </div>
58             )}
59           </div>
60           <div className="p-3 border-t flex gap-2">
61             <input value={input} onChange={e => setInput(e.target.value)}
62               onKeyDown={e => e.key === 'Enter' && sendMessage()}
63             className="flex-1 rounded-lg border px-3 py-1.5 text-sm"

```

```

64         placeholder="Posez une question a Melo..." />
65         <button onClick={sendMessage}
66             className="bg-indigo-600 text-white rounded-lg px-3 py-1.5 text-sm
67                 ↪ ">
68             Envoyer
69         </button>
70     </div>
71 </motion.div>
72 </AnimatePresence>
73 <button onClick={() => setOpen(!open)}
74     className="w-14 h-14 rounded-full bg-indigo-600 text-white shadow-lg
75         flex items-center justify-center hover:bg-indigo-700 transition"
76     ↪ >
77     <Bot size={28} />
78 </button>
79 </div>
80 );
81 };

```

8.3 Modèle de Scénario et Moteur d'Automatisation

8.3.1 Schéma Scénario

Listing 8.4 – Schéma Scénario (backend/models/Scenario.js)

```

1  const scenarioSchema = new mongoose.Schema({
2    name: { type: String, required: true },
3    description: { type: String },
4    houseCode: { type: String, required: true },
5    isActive: { type: Boolean, default: true },
6    triggerType: { type: String, enum: ['time'], default: 'time' },
7    scheduleTime: { type: String, required: true }, // Format "HH:MM"
8    action: { type: String, required: true }, // ex. "mood:Morning"
9    targetDevice: { type: mongoose.Schema.Types.ObjectId, ref: 'Device' },
10   lastTriggeredAt: { type: Date }
11 }, { timestamps: true });

```

8.3.2 Service Planificateur

Listing 8.5 – Planificateur de Scénarios (backend/scheduler.js)

```

1  const { schedule } = require('node-cron');
2
3  const initScheduler = (mqttClient, io) => {
4    // Exécuter chaque minute
5    schedule('* * * * *', async () => {
6      const now = new Date();
7      const currentTime = `${String(now.getHours()).padStart(2, '0')}:${`
8        + `${String(now.getMinutes()).padStart(2, '0')}`;
9
10     const scenarios = await Scenario.find({
11       isActive: true,
12       scheduleTime: currentTime
13     });
14
15     for (const scenario of scenarios) {
16       try {
17         if (scenario.action.startsWith('mood:')) {
18           const scene = scenario.action.split(':')[1];
19           await activateScene(scene, scenario.houseCode, mqttClient, io);
20         } else if (scenario.action === 'lock_all') {
21           await lockAllDoors(scenario.houseCode, mqttClient);
22         } else if (scenario.action.startsWith('toggle:light:')) {
23           const state = scenario.action.endsWith(':on') ? 'ON' : 'OFF';
24           await toggleLights(scenario.houseCode, state, mqttClient);
25         }
26         scenario.lastTriggeredAt = new Date();
27         await scenario.save();
28
29         io.to(scenario.houseCode).emit('scenario:triggered', {
30           name: scenario.name, action: scenario.action
31         });
32
33         await AuditLog.create({
34           actorName: 'Planificateur',
35           actorRole: 'system',
36           action: `Scenario "${scenario.name}" déclenche : ${scenario.action}`,
37           category: 'scenario',
38           houseCode: scenario.houseCode
39         });
40       } catch (err) {
41         console.error(`[Planificateur] Erreur scenario ${scenario._id}:`, err);
42       }
43     }
44   });
45 };

```

8.4 Surveillance Énergétique

8.4.1 Algorithme de Calcul de la Consommation

La consommation énergétique est estimée à partir de la collection AuditLog. Chaque événement ON pour un appareil marque le début d'une période de consommation. L'événement OFF correspondant (ou la fin de session) la ferme. Le kWh pour chaque période est :

$$\text{kWh} = \frac{W_{\text{appareil}} \times \Delta t_{\text{heures}}}{1000}$$

Listing 8.6 – Endpoint Énergie (backend/routes/energy.js)

```
1 router.get('/hourly', protect, async (req, res) => {
2   const WATTAGE = { light:10, lamp:10, 'rgb-light':10, ac:1500,
3     tv:80, fan:50, outlet:200, 'washing-machine':500 };
4
5   const logs = await AuditLog.find({
6     houseCode: req.user.houseCode,
7     category: 'device',
8     createdAt: { $gte: new Date(Date.now() - 86400000) } // Dernieres 24h
9   }).sort({ createdAt: 1 });
10
11   // Construire la carte de consommation horaire
12   const hourly = Array(24).fill(0);
13   const openPeriods = {}; // deviceId -> { startHour, watt }
14
15   for (const log of logs) {
16     const deviceId = log.metadata?.deviceId;
17     const action = log.action;
18     const hour = new Date(log.createdAt).getHours();
19     const watt = WATTAGE[log.metadata?.deviceType] ?? 10;
20
21     if (action.includes('ON') || action.includes('OPEN')) {
22       openPeriods[deviceId] = { startHour: hour, watt };
23     } else if (openPeriods[deviceId]) {
24       const duration = (hour - openPeriods[deviceId].startHour) || 1;
25       hourly[hour] += (openPeriods[deviceId].watt * duration) / 1000;
26       delete openPeriods[deviceId];
27     }
28   }
29
30   res.json({ hourly, totalKwh: hourly.reduce((a,b) => a+b, 0) });
31 });
```

8.5 Système de Demandes de Permission

8.5.1 Modèle DemandePermission

Listing 8.7 – Schéma DemandePermission (backend/models/PermissionRequest.js)

```
1 const permissionRequestSchema = new mongoose.Schema({
2   requester: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
3     ↪ },
4   houseAdmin: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true
5     ↪ },
6   actionKey: { type: String, required: true }, // ex. 'room:kitchen'
7   actionLabel: { type: String, required: true }, // libelle lisible
8   room: { type: String }, // piece demandee (optionnel)
9   status: { type: String,
10     enum: ['pending', 'approved', 'denied'], default: 'pending' },
11   reviewer: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
12   reviewNotes: { type: String },
13   adminReadAt: { type: Date },
14   requesterReadAt: { type: Date }
15 }, { timestamps: true });
```

8.5.2 Flux d'Approbation de Permission

Listing 8.8 – Approbation Demande de Permission (backend/routes/permissions.js)

```
1 router.post('/:id/approve', protect, admin, async (req, res) => {
2   const { reviewNotes } = req.body;
3   const request = await PermissionRequest.findById(req.params.id)
4     .populate('requester');
5
6   if (!request || request.status !== 'pending')
7     return res.status(400).json({ message: 'Demande introuvable ou deja traitee'
8       ↪ });
9
10  request.status = 'approved';
11  request.reviewer = req.user._id;
12  request.reviewNotes = reviewNotes;
13  await request.save();
14
15  // Accorder la permission au resident
16  const resident = await User.findById(request.requester._id);
17  if (!resident.permissions.includes(request.actionKey)) {
18    resident.permissions.push(request.actionKey);
19  }
```

```
18   }
19   if (request.room && !resident.room) {
20     resident.room = request.room;
21   }
22   await resident.save();
23
24   // Notifier le resident en temps reel
25   io.to('user:${resident._id}').emit('permission:updated', {
26     status: 'approved', actionKey: request.actionKey
27   });
28
29   res.json({ success: true });
30 };
```

8.6 Revue et Rétrospective Sprint 5

Revue Sprint 5

Complété : 4 stories sur 4 (34 PS). La démo Melo a impressionné.

Points forts : Taper “Allume toutes les lumières de la chambre à 80%” a fait illuminer la LED RGB chambre à 80% de luminosité en 1,5 secondes. Le graphique d’énergie a montré des pics de consommation réalistes lors du déclenchement du scénario climatisation.

Rétrospective :

- **Ce qui a bien fonctionné :** La latence d’inférence Groq (<800 ms) rend Melo très réactif.
- **Défi :** L’analyse des blocs ACTION a nécessité des regex soigneuses pour éviter les faux positifs dans les réponses conversationnelles.
- **Amélioration :** Ajout de `temperature: 0.3` pour réduire les noms d’appareils hallucités dans les réponses LLM.

Sprint 6 — Portail Agence, Journal d'Audit et Finalisation

Sprint 6 — Portail Agence et Audit

Durée : 2 semaines **Points de Story :** 37 **Objectif :** Livrer le hub conciergerie agence avec streaming en direct des maisons, un tableau de bord de journal d'audit complet, les demandes de permission résidents, les modes de sécurité et la gestion du profil.

9.1 Hub Conciergerie et Portail Agence

9.1.1 Server-Sent Events (SSE) pour le Statut en Direct

Listing 9.1 – Endpoint SSE pour le Hub Conciergerie (backend/routes/concierge.js)

```
1 router.get('/stream', protect, agencyOnly, (req, res) => {
2   res.setHeader('Content-Type', 'text/event-stream');
3   res.setHeader('Cache-Control', 'no-cache');
4   res.setHeader('Connection', 'keep-alive');
5   res.flushHeaders();
6
7   const push = (data) => res.write(`data: ${JSON.stringify(data)}\n\n`);
8
9   // Envoyer l'instantane initial de tous les statuts admin
10  push({ type: 'snapshot', houses: getOnlineAdmins() });
11
12  // S'abonner aux evenements de connexion admin
13  const onConnect = (admin) => push({ type: 'online', houseCode: admin.houseCode
    ↪ });
```

```

14   const onDisconnect = (admin) => push({ type: 'offline', houseCode: admin.
      ↪ houseCode });
15
16   adminEvents.on('connect', onConnect);
17   adminEvents.on('disconnect', onDisconnect);
18
19   // Nettoyage a la deconnexion du client
20   req.on('close', () => {
21     adminEvents.off('connect', onConnect);
22     adminEvents.off('disconnect', onDisconnect);
23     res.end();
24   });
25 });

```

9.2 Système de Journal d'Audit

9.2.1 Schéma AuditLog

Listing 9.2 – Schéma AuditLog (backend/models/AuditLog.js)

```

1  const auditLogSchema = new mongoose.Schema({
2    actor: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
3    actorName: { type: String, required: true },
4    actorRole: { type: String },
5    action: { type: String, required: true },
6    category: { type: String,
7      enum: ['auth', 'device', 'security', 'user', 'scenario', 'system'],
8      default: 'system' },
9    details: { type: String },
10   metadata: { type: mongoose.Schema.Types.Mixed },
11   ipAddress: { type: String },
12   houseCode: { type: String }
13 }, { timestamps: true });

```

9.2.2 Événements Journalisés

Table 9.1 – Catégories et Événements du Journal d'Audit

Catégorie	Événements Journalisés
auth	Connexion (succès/échec), inscription, réinitialisation mdp, vérification email, 2FA activer/désactiver, connexion Google OAuth
device	Appareil créé, mis à jour, supprimé, commande exécutée (ON/OFF/luminosité/scène)
security	Urgence gaz activée, urgence levée, mode Absence activé, mode Verrouillage activé
user	Résident ajouté/supprimé, rôle modifié, permission accordée/révoquée
scenario	Scénario créé/mis à jour/supprimé, scénario déclenché par le planificateur
system	Démarrage serveur, démarrage courtier MQTT, initialisation planificateur

9.3 Modes de Sécurité

Table 9.2 – Définition des Modes de Sécurité

Mode	Comportement
Normal	Fonctionnement standard ; tous les appareils librement contrôlables par les utilisateurs autorisés.
Mode Absence	Surveillance caméra renforcée ; toutes les portes extérieures verrouillées ; fenêtres fermées ; vanne gaz fermée ; événements de mouvement journalisés.
Mode Verrouillage	Sécurité maximale ; toutes les caméras armées ; toutes les portes et fenêtres verrouillées ; vanne gaz fermée ; contrôle des appareils résidents suspendu.

9.4 Récapitulatif Complet des Routes API

Table 9.3 – Toutes les Routes Backend API

Préfixe de Route	Auth	Rôle
/api/auth	Aucune/JWT	Inscription, connexion, vérification email, ré-initialisation mdp
/api/oauth	Aucune	Callback Google OAuth 2.0
/api/2fa	JWT	Configuration, vérification, désactivation TOTP 2FA
/api/profile	JWT	Obtenir/mettre à jour le profil et l'avatar utilisateur
/api/users	Admin JWT	Liste, mise à jour, suppression des utilisateurs de la maison
/api/devices	Admin JWT	CRUD des appareils
/api/floorplan	JWT	État plan, commandes, scènes
/api/scenarios	Admin JWT	CRUD des scénarios
/api/gas	Admin JWT	État gaz, seuil, levée urgence
/api/permissions	JWT	CRUD demandes de permission et approbation
/api/messages	JWT	Soumission de message support et réponse admin
/api/agency	Agence JWT	Liste des admins pour la vue agence
/api/units	Admin JWT	Gestion des unités/maisons
/api/concierge	Agence JWT	Données hub conciergerie et flux SSE
/api/audit	Admin JWT	Requête et pagination du journal d'audit
/api/stats	Admin JWT	Statistiques tableau de bord
/api/energy	Admin JWT	Consommation énergétique horaire/quotidienne
/api/companion	JWT	Endpoint chat assistant IA
/api/owner-requests	Aucune/JWT	Demandes de compte propriétaire

9.5 Revue et Rétrospective Sprint 6

Revue Sprint 6

Complété : 8 stories sur 8 (37 PS). Plateforme fonctionnellement complète.

Points forts : Le hub conciergerie agence a affiché 3 maisons simulées avec le clignotement en ligne/hors ligne en direct. Le journal d'audit a été exporté avec succès en PDF avec jsPDF. Le mode Absence a immédiatement verrouillé tous les servomoteurs de porte sur l'ESP32.

Rétrospective :

- **Ce qui a bien fonctionné :** SSE était plus simple que Socket.IO pour le cas d'utilisation de streaming en lecture seule de l'agence.
- **Défi :** Le formatage de l'export PDF a nécessité un réglage minutieux des largeurs de colonnes dans jsPDF pour les longues chaînes d'action d'audit.
- **Amélioration :** Ajout d'une limite de 2 Mo sur les uploads d'avatar après qu'une image base64 de 15 Mo a causé des avertissements de taille de document MongoDB.

Sécurité, Tests et Assurance Qualité

10.1 Architecture de Sécurité

La Plateforme SmartHome implémente une stratégie de sécurité en **défense en profondeur** couvrant la sécurité du transport, l'authentification, l'autorisation, la validation des entrées et la journalisation d'audit.

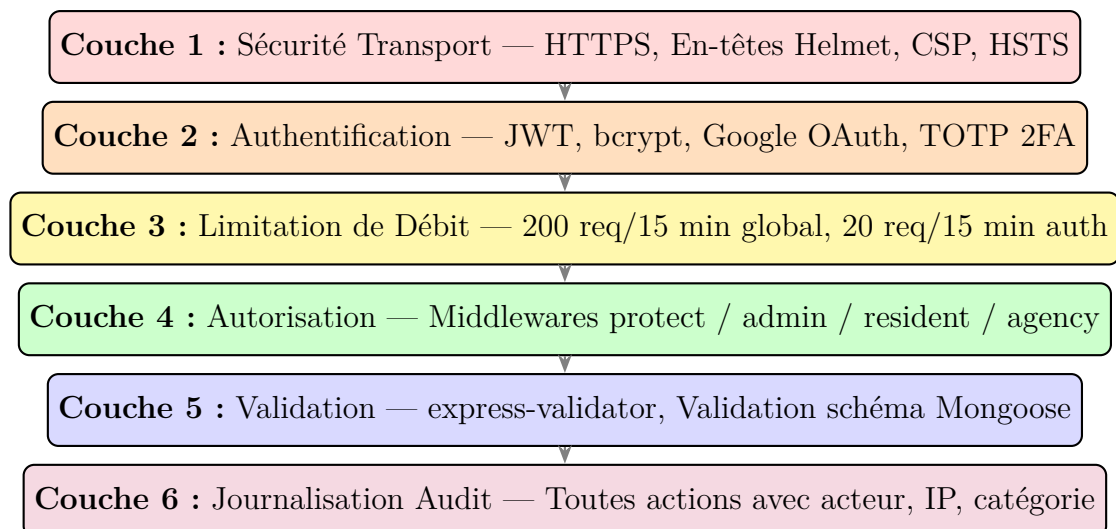


Figure 10.1 – Couches de Sécurité en Défense en Profondeur

10.2 Conformité OWASP Top 10

Table 10.1 – Conformité OWASP Top 10 (2021)

ID	Risque	Statut	Mitigation
A01	Contrôle d'Accès Défaillant	Mitigé	Middleware de rôle, scopage codeMaison sur toutes les requêtes
A02	Défaillances Cryptographiques	Mitigé	bcrypt pour mdp, JWT HS256, HTTPS en production
A03	Injection	Mitigé	Requêtes paramétrées Mongoose, express-validator sur toutes les entrées
A04	Conception Non Sécurisée	Mitigé	Couches de défense en profondeur, déclencheur de sécurité local ESP32
A05	Mauvaise Configuration Sécurité	Mitigé	En-têtes Helmet, liste blanche CORS, secrets dans .env
A06	Composants Vulnérables	Partiel	Dépendances maintenues ; pas de scanner CVE automatisé en CI
A07	Échecs Authn et Session	Mitigé	Expiration JWT, limitation de débit, support 2FA, bcrypt
A08	Intégrité Logicielle	Partiel	Pas de validation supply-chain en CI ; package-lock.json épinglé
A09	Échecs de Journalisation	Mitigé	Collection AuditLog pour toutes les actions, filtrage par catégorie
A10	SSRF	Mitigé	Aucune récupération d'URL contrôlée par l'utilisateur dans le backend

10.3 Stratégie de Tests

Table 10.2 – Stratégie de Tests par Niveau

Niveau	Outils	Couverture
Tests Unitaires	Jest / manuel	Fonctions utilitaires (génération code, formule PPM, utils JWT)
Tests d'Intégration	Postman	19 groupes de routes API ; cas nominal + cas d'erreur
Tests de Bout en Bout	Manuel (navigateur)	Flux complets : inscription → connexion → contrôle → urgence gaz → IA

Niveau	Outils	Couverture
Tests Matériels	MQTT Explorer	Timing servo ESP32, PWM LED, routage topics MQTT, lectures ADC capteur

10.3.1 Cas de Test Principaux

Table 10.3 – Cas de Test d’Intégration Sélectionnés

CT-ID	Cas de Test	Résultat At-	Résultat tendu
CT-01	Inscription Admin sans caractère spécial dans le mdp	HTTP 400 + erreur validation	✓Réussi
CT-02	Connexion avec identifiants valides	HTTP 200 + token JWT	✓Réussi
CT-03	Accès route protégée sans token	HTTP 401	✓Réussi
CT-04	Accès route admin en tant que résident	HTTP 403	✓Réussi
CT-05	Créer appareil avec topic MQTT dupliqué	HTTP 400 + erreur unicité	✓Réussi
CT-06	Publier commande appareil et vérifier livraison MQTT à l’ESP32	LED physique s’allume	✓Réussi
CT-07	Activer scène Matin et vérifier ouverture servo fenêtre	Servo à 180°	✓Réussi
CT-08	Publier PPM gaz au-dessus du seuil via MQTT	Mode urgence activé	✓Réussi
CT-09	Admin lève urgence ; vérifier buzzer OFF et vanne OUVERTE	Commandes publiées	✓Réussi
CT-10	Melo IA : commande “Éteins la lumière cuisine”	État lumière = OFF, MQTT publié	✓Réussi
CT-11	Résident demande permission pour room :cuisine	Statut : en attente, admin notifié	✓Réussi
CT-12	Admin approuve demande ; résident reçoit notification TR	Événement socket émis	✓Réussi
CT-13	Limite de débit : 21ème requête auth en 15 min	HTTP 429	✓Réussi

CT-ID	Cas de Test	Résultat attendu	At-	Résultat
CT-14	Upload image avatar de 16 Mo	HTTP 400 (limite taille)	(li-	✓Réussi
CT-15	Scénario déclenché à l'heure HH :MM correcte	Appareils mis à jour, entrée audit		✓Réussi

10.4 Limitations Connues

Table 10.4 – Limitations Connues et Mitigations

Limitation	Sévérité	Mitigation / Travaux Futurs
JWT non révocable avant expiration	Moyenne	Expiration courte planifiée ; table de liste noire (futur)
Courtier MQTT sans TLS en dev	Moyenne	Certificats TLS prévus pour le déploiement production
Stockage avatar base64 dans MongoDB	Faible	Migrer vers S3/MinIO (futur)
Pas de pipeline CI/CD automatisé	Faible	Tests Postman manuels avant chaque démo de sprint
Calibration capteur MQ6 variable	Moyenne	Valeur de calibration Ro (9,83) mesurée par unité
Backend instance unique (pas HA)	Faible	Conception sans état supporte la mise à l'échelle horizontale

Gestion de Projet, Planification et Gouvernance des Risques

11.1 Mise en Œuvre du Cadre Scrum

11.1.1 Rôles et Cérémonies

Le cadre Scrum a été adapté au contexte académique à développeur unique comme suit :

Table 11.1 – Correspondance des Rôles Scrum

Rôle Scrum	Personne	Responsabilités dans ce Projet
Product Owner	Encadrant Académique	Valide les exigences, priorise le backlog, fournit le feedback lors des revues de sprint
Scrum Master	Houssem E. Abdelal	Facilite les cérémonies, lève les impedi-ments, suit la vélocité
Équipe Dev	Houssem E. Abdelal	Développement full-stack, firmware, tests, documentation

11.1.2 Cérémonies de Sprint

Chaque sprint de 2 semaines comprend les cérémonies suivantes :

Planification de Sprint (Jour 1, 1h) : Revue du backlog produit, sélection des user stories, décomposition en tâches d'implémentation, estimation en heures.

Stand-up Quotidien (Chaque jour, 15 min) : Revue de la progression de la veille, plan du jour et impediments (adapté en format journal auto-géré).

Revue de Sprint (Jour 14, 30 min) : Démonstration des fonctionnalités complètes à l'encadrant, collecte des retours, mise à jour du backlog.

Rétrospective de Sprint (Jour 14, 15 min) : Identifier ce qui a bien fonctionné, ce qui peut être amélioré, et les actions pour le sprint suivant.

11.2 Chronogramme du Projet

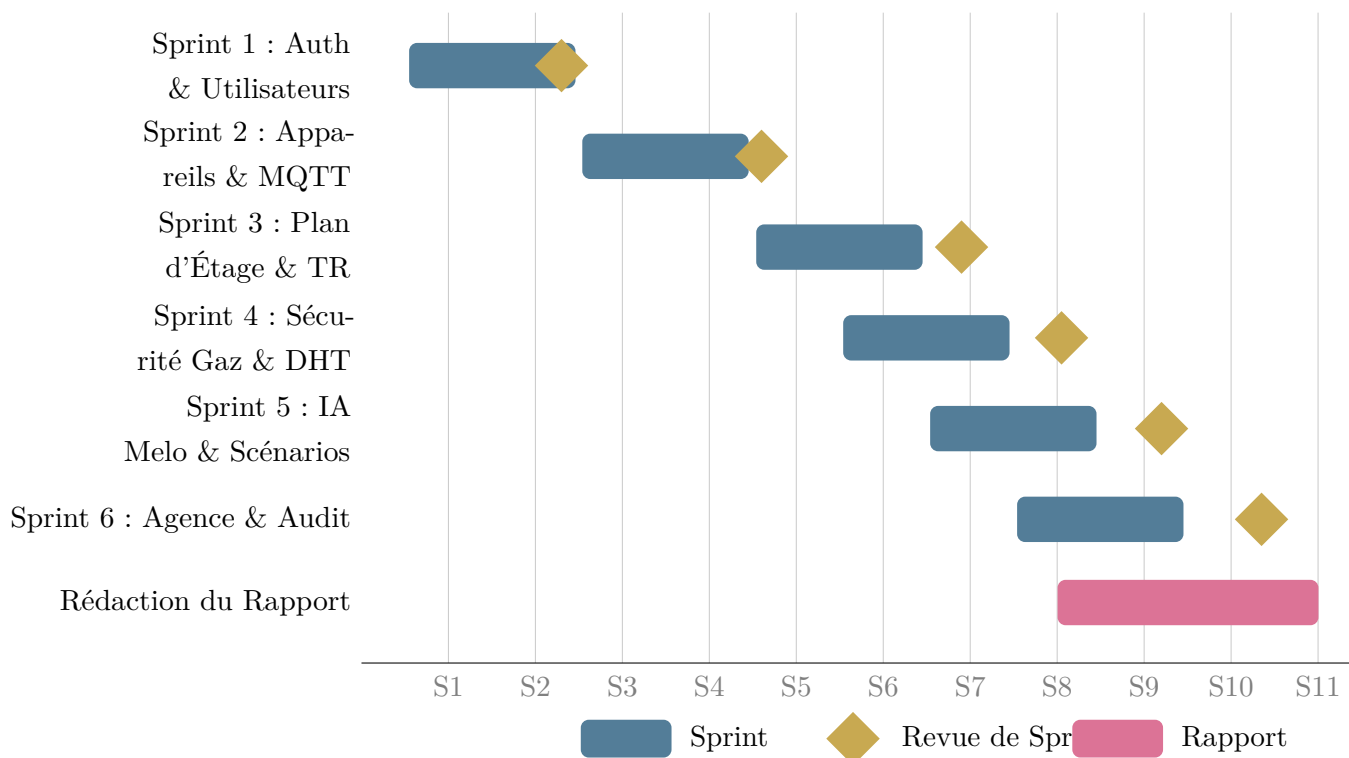


Figure 11.1 – Diagramme de Gantt du Projet (12 Semaines)

11.3 Vitesse des Sprints

Table 11.2 – Suivi de la Vitesse par Sprint

Sprint	PS Prévus	PS Réalisés	Vitesse
1	34	34	100%
2	39	39	100%
3	37	37	100%
4	36	36	100%
5	34	34	100%
6	37	37	100%
Total	217	217	100%

11.4 Registre des Risques

Table 11.3 – Registre des Risques du Projet

ID	Risque	Probabilité	Impact	Stratégie de Mitigation
R1	Déconnexion Wi-Fi ESP32 causant une perte de commandes	Haute	Haute	MQTT QoS 1 + reconnexion automatique firmware avec backoff 5 s
R2	Limites du plan gratuit MongoDB Atlas	Moyenne	Moyenne	MongoDB local en secours ; chaîne de connexion dans <code>.env</code>
R3	Expiration ou limite de débit de l'API Groq	Faible	Moyenne	Message d'erreur gracieux dans Melo ; limite 10 req/min par utilisateur
R4	Capteur DHT11 retournant NaN lors des lectures à froid	Haute	Faible	Guard <code>isnan()</code> avant publication MQTT ; ignorer les lectures erronées
R5	WebSocket bloqué par certains réseaux	Faible	Moyenne	Fallback automatique Socket.IO vers HTTP long-polling
R6	Exposition du secret JWT dans un dépôt public	Faible	Critique	<code>.env</code> dans <code>.gitignore</code> ; <code>.env.example</code> committé à la place

ID	Risque	Probabilité	Impact	Stratégie de Mitigation
R7	Extension non planifiée du périmètre	Moyenne	Moyenne	Backlog produit strict ; approbation de l'encadrant pour tout changement
R8	Faux positifs du capteur MQ6 lors de la cuisine	Moyenne	Moyenne	Seuil configurable (défaut 400 PPM) ; acquittement manuel admin
R9	Décalage de fuseau horaire dans les scénarios cron	Faible	Moyenne	Heure système synchronisée NTP ; toutes les dates en UTC dans MongoDB
R10	Développeur unique — point de défaillance unique	Haute	Moyenne	Documentation complète et run-book pour la continuation du projet

11.5 Métriques Récapitulatives du Projet

Métrique	Valeur
Durée totale du projet	12 semaines
Nombre de sprints	6 (2 semaines chacun)
Points de story livrés	217 PS
Vélocité moyenne	36 PS / sprint
Fichiers de routes backend	19
Collections de base de données	9
Composants de pages frontend	34
Composants réutilisables frontend	25+
Lignes de code firmware ESP32	≈430
Endpoints API	≈75
Cas de test Postman documentés	15
Risques OWASP Top 10 mitigés	8/10 entièrement, 2/10 partiellement

Conclusion et Perspectives

12.1 Conclusion du Projet

Ce rapport de PFE a présenté la conception, l'implémentation et la validation complètes de la **Plateforme IoT Maison Intelligente** — un système unifié d'automatisation résidentielle de qualité production, développé en six sprints Scrum sur l'année académique 2025–2026 à l'ISET Siliana.

La plateforme répond avec succès à tous les onze objectifs définis au Chapitre 1 :

- **Authentification multi-rôles (O1)** : Cinq mécanismes d'authentification (email/mot de passe, Google OAuth, 2FA, vérification email, réinitialisation) pour quatre rôles.
- **Gestion des appareils (O2)** : Cycle de vie CRUD complet pour 20+ types d'appareils avec routage MQTT par topic unique.
- **Intégration MQTT (O3)** : Courtier Aedes en processus éliminant tout middleware externe ; latence bout en bout inférieure à 500 ms sur réseau local.
- **Synchronisation temps réel (O4)** : Passerelle Socket.IO avec chambres scopées par code maison assurant l'isolation multi-locataire.
- **Sécurité gaz (O5)** : Protocole d'urgence à deux couches (déclencheur ADC local ESP32 + seuil serveur PPM) avec contrôle matériel vanne et buzzer.
- **Workflow de permissions (O6)** : Demandes granulaires des résidents avec notifications admin en temps réel.
- **Plan d'étage 3D (O7)** : Plan isométrique SVG avec overlays de pièces cliquables et interpolation de couleur d'éclairage dynamique.

- **Assistant IA Melo (O8)** : Alimenté par Groq/Llama 3, respectant le contrôle d'accès basé sur les rôles.
- **Automatisation par scénarios (O9)** : Planificateur cron temporel exécutant des préréglages d'ambiance et des opérations de verrouillage.
- **Surveillance énergétique (O10)** : Estimation kWh horaire depuis les journaux d'audit avec visualisation recharts.
- **Déploiement (O11)** : Configuration d'environnement documentée et instructions de flashage ESP32.

12.1.1 Leçons Apprises

1. **La séparation des protocoles est essentielle** : Utiliser MQTT pour l'IoT et Socket.IO pour les navigateurs avec le backend comme traducteur s'est révélé bien plus propre que de tenter de faire fonctionner un seul protocole dans les deux domaines.
2. **La défense en profondeur pour les systèmes critiques** : La réponse d'urgence gaz à deux couches (firmware + serveur) a démontré que la logique de sécurité ne doit jamais reposer sur un seul canal de communication.
3. **La vélocité agile bénéficie aux projets solo** : La livraison sprint par sprint a permis un feedback d'intégration matérielle précoce qui aurait été manqué dans une approche de développement monolithique.
4. **Les contraintes LLM sont aussi importantes que les capacités** : Limiter l'assistant IA au respect du contrôle d'accès basé sur les rôles était aussi complexe que l'intégration LLM elle-même.
5. **Le journal d'audit est une fonctionnalité, pas une réflexion après coup** : La collection AuditLog est devenue le fondement de la surveillance énergétique, du suivi des scénarios et de la supervision de l'agence.

12.2 Perspectives et Travaux Futurs

12.2.1 Améliorations à Court Terme (3 prochains mois)

- **MQTT TLS** : Activer TLS sur le courtier Aedes (port 8883) et mettre à jour le firmware ESP32 pour utiliser `WiFiClientSecure`.

- **Révocation de token JWT** : Implémenter une liste noire de tokens basée sur Redis pour l'invalidation immédiate de session.
- **Stockage objet pour avatars** : Migrer le stockage base64 vers AWS S3 ou MinIO pour maintenir des tailles de documents MongoDB raisonnables.
- **Pipeline CI/CD** : Ajouter un workflow GitHub Actions pour les vérifications ESLint automatisées, les tests Jest et la construction d'images Docker.
- **Application mobile** : Construire une application React Native ou Flutter consommant le même backend REST + Socket.IO.

12.2.2 Fonctionnalités à Moyen Terme (3–6 mois)

- **Contrôle vocal** : Intégrer l'API Web Speech dans le navigateur pour le contrôle mains libres des appareils.
- **Streaming caméra en direct** : Intégrer des flux RTSP ou WebRTC directement dans l'overlay du plan d'étage via un serveur média (ex. : MediaMTX).
- **Scénarios conditionnels** : Étendre le modèle de scénario pour supporter des déclencheurs basés sur des conditions (ex. : niveau gaz, température).
- **Base de données time-series** : Migrer la télémétrie des capteurs vers TimescaleDB ou InfluxDB pour un stockage long terme efficace.
- **Support multi-étages** : Étendre le plan d'étage pour supporter plusieurs niveaux avec un sélecteur d'étage.

12.2.3 Directions de Recherche à Long Terme (6+ mois)

- **Détection d'anomalies ML** : Entraîner un modèle de détection d'anomalies sur les patterns de consommation énergétique pour détecter automatiquement les comportements inhabituels des appareils.
- **Protocole Matter/Thread** : Migrer la communication des appareils vers le standard ouvert **Matter** pour l'interopérabilité avec les écosystèmes commerciaux (Apple HomeKit, Google Home, Amazon Alexa).
- **Calcul en périphérie** : Exécuter l'inférence pour la détection d'anomalies gaz et les réponses de l'assistant IA localement sur un ESP32-S3 avec TensorFlow Lite Micro.
- **Audit blockchain** : Stocker les hachages du journal d'audit sur une blockchain permissionnée pour des enregistrements opérationnels inviolables dans les propriétés réglementées.

12.3 Remarques Finales

La Plateforme IoT Maison Intelligente démontre qu'une équipe d'ingénierie réduite et focalisée peut livrer un produit IoT de qualité production, riche en fonctionnalités, lorsqu'elle dispose d'outils open source modernes et d'un processus agile discipliné.

La plateforme n'est pas seulement une preuve de concept : c'est un système déployable avec une intégration matérielle réelle, des contrôles de sécurité réels et une expérience utilisateur réelle qui peut être étendue et exploitée dans un contexte résidentiel dès aujourd'hui.

La méthodologie Scrum s'est révélée essentielle pour gérer la complexité et maintenir l'élan sur 12 semaines, permettant un feedback continu des parties prenantes et une livraison cohérente. Les choix architecturaux — événementiel, en couches et à protocoles séparés — créent une fondation qui peut évoluer horizontalement et absorber les exigences futures sans refonte architecturale.

Récapitulatif du Projet

217 points de story livrés sur 6 sprints en 12 semaines

Plateforme IoT full-stack : **Node.js + React + MongoDB + MQTT + ESP32 + Groq IA**

100% des user stories planifiées complétées

8/10 risques OWASP Top 10 entièrement mitigés

Annexe A : Référence Complète de l'API

13.1 Vue d'Ensemble

Tous les endpoints API utilisent **JSON** pour les corps de requête et de réponse. Les endpoints protégés nécessitent un token Bearer dans l'en-tête **Authorization**.

Listing 13.1 – En-têtes de Requête Communs

```
1 Content-Type: application/json
2 Authorization: Bearer <jwt_token>
```

13.2 Endpoints d'Authentification (/api/auth)

Table 13.1 – API Authentification

Méthode	Chemin	Corps / Réponse
POST	/api/auth/register/admin	{nom,email,motdepasse} → {token,user,codeMaison}
POST	/api/auth/register/resident	{nom,email,mdp,codeMaison} → {token,user}
POST	/api/auth/register/agency	{nom,email,mdp,codeAgence} → {token,user}
POST	/api/auth/login	{email,motdepasse,totpToken?} → {token,user}

Méthode	Chemin	Corps / Réponse
GET	/api/auth/verify/:token	Vérifie l'email, redirige vers le frontend
POST	/api/auth/forgot-password	{email} → {message}
POST	/api/auth/reset-password	{token,motdepasse} → {message}
GET	/api/auth/me	(JWT) → objet utilisateur complet
POST	/api/auth/resent-verification	{email} → {message}

13.3 Endpoints Appareils (/api/devices)

Table 13.2 – API Appareils

Méthode	Chemin	Corps / Réponse
GET	/api/devices	(Admin JWT) → liste [Appareil] de la maison
POST	/api/devices	{nom,type,topicMQTT,pièce,...} → Appareil
PUT	/api/devices/:id	Mise à jour partielle → Appareil mis à jour
DELETE	/api/devices/:id	Supprime l'appareil, retourne {success:true}
GET	/api/devices/:id	Appareil unique par ID
POST	/api/devices/:id/command	{command:{state,brightness?}} → {success}

13.4 Endpoints Plan d'Étage (/api/floorplan)

Table 13.3 – API Plan d'Étage

Méthode	Chemin	Corps / Réponse
GET	/api/floorplan/state	(JWT) → {appareils:[],houseState}
POST	/api/floorplan/command	{deviceId,command} → {success}
POST	/api/floorplan/scene	{scene:"Morning" "Dinner" "Cinematic" "Sleep"} → {success,scene}

Méthode	Chemin	Corps / Réponse
GET	/api/floorplan/house-state	(JWT) → document <code>HouseState</code>
POST	/api/floorplan/sync-defaults	(Admin) → synchronise les positions d'appareils

13.5 Endpoints Sécurité Gaz (/api/gas)

Table 13.4 – API Gaz

Méthode	Chemin	Corps / Réponse
GET	/api/gas/state	(JWT) → {niveau, seuil, vanneOuverte, urgence}
POST	/api/gas/clear-emergency	(Admin JWT) → {success:true}
POST	/api/gas/set-threshold	(Admin JWT) {threshold:400} → {success}

13.6 Endpoint Compagnon IA (/api/companion)

Table 13.5 – API Compagnon

Méthode	Chemin	Corps / Réponse
POST	/api/companion/chat	{messages:[role,content]} → {reply,actionsExecuted}

13.7 Endpoints Scénarios (/api/scenarios)

Table 13.6 – API Scénarios

Méthode	Chemin	Corps / Réponse
GET	/api/scenarios	(Admin JWT) → liste des scénarios de la maison
POST	/api/scenarios	{nom,action,scheduleTime,isActive} → Scénario
PUT	/api/scenarios/ :id	Mise à jour partielle → Scénario mis à jour
DELETE	/api/scenarios/ :id	Supprime le scénario → {success:true}

13.8 Codes de Statut HTTP Courants

Code	Signification dans cette API
200	OK — requête traitée avec succès
201	Créé — nouvelle ressource créée
400	Mauvaise Requête — échec de validation, doublon ou entrée invalide
401	Non Autorisé — token JWT manquant ou invalide
403	Interdit — authentifié mais rôle/permissions insuffisants
404	Introuvable — la ressource n'existe pas
429	Trop de Requêtes — limite de débit dépassée
500	Erreur Interne Serveur — défaillance inattendue du serveur

Annexe B : Architecture Frontend et Interface Utilisateur

14.1 Routage de l'Application

L'application React utilise **React Router v7** avec des gardes de routes basées sur les rôles. Les routes sont protégées par l'AuthContext qui lit le JWT depuis localStorage et expose l'objet utilisateur courant à tous les composants.

Listing 14.1 – Configuration des Routes (frontend/src/App.jsx)

```
1 <Routes>
2   {/* Routes publiques */}
3   <Route path="/" element={<Home />} />
4   <Route path="/auth" element={<AuthEntry />} />
5   <Route path="/admin/login" element={<AdminLogin />} />
6   <Route path="/admin/register" element={<AdminRegister />} />
7   <Route path="/resident/login" element={<ResidentLogin />} />
8   <Route path="/oauth/callback" element={<OAuthCallback />} />
9   <Route path="/verify-email" element={<VerifyEmail />} />
10  <Route path="/reset-password" element={<ResetPassword />} />
11
12  {/* Protegees : routes Admin */}
13  <Route element={<AdminGuard />>
14    <Route path="/dashboard" element={<Dashboard />} />
15    <Route path="/devices" element={<Devices />} />
16    <Route path="/rooms" element={<Rooms />} />
17    <Route path="/scenarios" element={<Scenarios />} />
18    <Route path="/security" element={<Security />} />
19    <Route path="/energy" element={<Energy />} />
20    <Route path="/users" element={<UserManagement />} />
21    <Route path="/admin-requests" element={<AdminRequests />} />
```

```

22   <Route path="/audit" element={<AuditLog />} />
23   <Route path="/messages" element={<Messages />} />
24 </Route>
25
26 { /* Protegees : routes Resident */}
27 <Route element={<ResidentGuard />}>
28   <Route path="/resident/dashboard" element={<ResidentDashboard />} />
29   <Route path="/resident/room" element={<ResidentRoom />} />
30   <Route path="/my-requests" element={<MyRequests />} />
31 </Route>
32
33 { /* Protegees : routes Agence */}
34 <Route element={<AgencyGuard />}>
35   <Route path="/agency/dashboard" element={<AgencyDashboard />} />
36 </Route>
37
38 { /* Speciales */}
39 <Route path="/concierge-hub" element={<ConciergeHub />} />
40 <Route path="/profile" element={<Profile />} />
41 <Route path="*" element={<NotFound />} />
42 </Routes>

```

14.2 Inventaire des Pages

Table 14.1 – Composants de Pages Frontend

Composant	Route	Rôle
Home	/	Page d'accueil marketing
AuthEntry	/auth	Sélecteur de rôle (Admin/Résident/Agence)
AdminLogin	/admin/login	Connexion admin email/mot de passe
AdminRegister	/admin/register	Inscription admin avec génération de code
ResidentLogin	/resident/login	Connexion résident
ResidentRegister	/resident/register	Inscription résident avec code maison
AgencyLogin	/agency/login	Connexion agence avec code agence
OAuthCallback	/oauth/callback	Gestionnaire de token Google OAuth
VerifyEmail	/verify-email	Confirmation de vérification email
ForgotPassword	/forgot	Demande d'email de réinitialisation mdp

Composant	Route	Rôle
ResetPassword	/reset-password	Formulaire de nouveau mot de passe
Dashboard	/dashboard	Tableau de bord admin (stats, plan, scènes)
ResidentDashboard	/resident/...	Tableau de bord simplifié résident
AgencyDashboard	/agency/...	Vue multi-unités agence
Devices	/devices	Liste complète des appareils et CRUD
Rooms	/rooms	Contrôle des appareils par pièce
ResidentRoom	/resident/room	Vue de la pièce assignée au résident
Scenarios	/scenarios	Éditeur d'automatisations temporelles
Security	/security	Surveillance caméra, modes Absence/Verrouillage
Energy	/energy	Graphiques de consommation énergétique
UserManagement	/users	Liste admin, ajout/suppression de résidents
AdminRequests	/admin-requests	Panneau d'approbation des demandes de permission
AuditLog	/audit	Journal d'audit filtrable avec export PDF
Messages	/messages	Gestion des tickets support
ConciergeHub	/concierge-hub	Surveillance en direct des maisons (agence)
Profile	/profile	Paramètres utilisateur, avatar, 2FA
MyRequests	/my-requests	Historique des demandes de permission résident
UnitOnboarding	/onboarding	Flux de configuration maison
NotFound	/*	Page 404

14.3 Composants Réutilisables Clés

Table 14.2 – Composants Réutilisables Clés

Composant	Rôle
FloorPlan	Plan SVG isométrique 3D avec tuiles de pièces interactives
DeviceCard	Carte de contrôle individuelle avec état et slider de luminosité
SmartDeviceOverlay	Panneau de contrôle popover ancré à l’icône du plan d’étage
GasEmergencyOverlay	Alerte d’urgence plein écran rouge avec bouton de levée
GasSafetyBanner	Bannière supérieure affichant le PPM gaz et l’état de la vanne
CameraMonitoringPanel	Grille de flux caméra avec indicateurs de détection de mouvement
AuraCompanion	Widget de chat IA flottant avec le personnage Melo
Melo3D	Mascotte IA 3D animée (React Three Fiber)
PermissionNotification	Icône cloche avec dropdown des demandes de permission temps réel
RoomSection	Groupe de pièce rétractable pour la page /rooms
ScenarioCard	Carte de pré-réglage de scène avec bouton d’activation
PasswordStrengthBar	Visualisation de force du mot de passe en temps réel (4 niveaux)
GoogleAuthButton	Bouton stylisé d’initiation Google OAuth
ConfirmDialog	Modal de confirmation réutilisable pour actions destructives
AppErrorBoundary	Boundary d’erreur React enveloppant l’application complète

14.4 Implémentation AuthContext

Listing 14.2 – AuthContext (frontend/src/context/AuthContext.jsx)

```

1  const AuthContext = createContext(null);
2
3  export const AuthProvider = ({ children }) => {
4    const [user, setUser] = useState(null);

```

```

5   const [token, setToken] = useState(
6     () => localStorage.getItem('token')
7   );
8
9   // Charger l'utilisateur si token present
10  useEffect(() => {
11    if (!token) return;
12    axios.get('/api/auth/me', {
13      headers: { Authorization: 'Bearer ${token}' }
14    })
15      .then(({ data }) => setUser(data.user))
16      .catch(() => {
17        setToken(null);
18        localStorage.removeItem('token');
19      });
20  }, [token]);
21
22  const login = (newToken, newUser) => {
23    localStorage.setItem('token', newToken);
24    setToken(newToken);
25    setUser(newUser);
26  };
27
28  const logout = () => {
29    localStorage.removeItem('token');
30    setToken(null);
31    setUser(null);
32  };
33
34  return (
35    <AuthContext.Provider value={{ user, token, login, logout }}>
36      {children}
37    </AuthContext.Provider>
38  );
39 };
40
41 export const useAuth = () => useContext(AuthContext);

```

14.5 ThemeContext et Mode Sombre

L'application supporte le basculement thème sombre/clair via la stratégie de classe dark de **Tailwind CSS**. Le thème sélectionné est stocké dans `localStorage` et appliqué

en basculant `document.documentElement.classList`.

Listing 14.3 – Implémentation ThemeContext (frontend/src/context/ThemeContext.jsx)

```
1  const ThemeContext = createContext(null);
2
3  export const ThemeProvider = ({ children }) => {
4    const [dark, setDark] = useState(
5      () => localStorage.getItem('theme') === 'dark'
6    );
7
8    useEffect(() => {
9      document.documentElement.classList.toggle('dark', dark);
10     localStorage.setItem('theme', dark ? 'dark' : 'light');
11   }, [dark]);
12
13   return (
14     <ThemeContext.Provider value={{ dark, toggle: () => setDark(d => !d) }}>
15       {children}
16     </ThemeContext.Provider>
17   );
18 };
```

Annexe C : Guide d'Installation et Runbook Opérationnel

15.1 Prérequis

Logiciel	Version	Rôle
Node.js	≥ 18 LTS	Backend et build frontend
npm	≥ 9	Gestion des paquets
MongoDB	≥ 6 (ou Atlas)	Base de données principale
Arduino IDE	2.x	Développement firmware ESP32
Git	≥ 2.40	Contrôle de version

15.2 Variables d'Environnement Backend

Créer `backend/.env` avec les variables suivantes :

Listing 15.1 – `backend/.env.example`

```
1 # Serveur
2 PORT=5000
3 MQTT_PORT=1883
4 NODE_ENV=development
5
6 # Base de donnees
7 MONGODB_URI=mongodb://localhost:27017/smarthome
8
9 # Securite
```

```
10 JWT_SECRET=votre-secret-aleatoire-256-bits
11 BCRYPT_ROUNDS=10
12
13 # Google OAuth 2.0
14 GOOGLE_CLIENT_ID=votre-google-client-id
15 GOOGLE_CLIENT_SECRET=votre-google-client-secret
16
17 # IA Groq (optionnel, Melo desactive si absent)
18 GROQ_API_KEY=gsk_votre-cle-api-groq
19
20 # Email (Nodemailer - exemple Gmail)
21 EMAIL_HOST=smtp.gmail.com
22 EMAIL_PORT=587
23 EMAIL_USER=votre-email@gmail.com
24 EMAIL_PASS=votre-mot-de-passe-application
25
26 # CORS
27 FRONTEND_URL=http://localhost:5173
28 BACKEND_URL=http://localhost:5000
29
30 # Codes d'accès
31 ADMIN_ACCESS_CODE=ADMIN2025
32 AGENCY_ACCESS_CODE=AGENCY2025
33 CONCIERGE_CODE=CONCIERGE2025
```

15.3 Étapes d'Installation Complètes

Listing 15.2 – Commandes d'Installation

```
1 # 1. Cloner le depot
2 git clone https://github.com/username/SmartHome-PFE.git
3 cd SmartHome-PFE
4
5 # 2. Installer toutes les dependances Node.js
6 npm run install:all
7 # Equivalent a :
8 # npm install (racine)
9 # cd backend && npm install (backend)
10 # cd ../frontend && npm install (frontend)
11
12 # 3. Configurer l'environnement
13 cp backend/.env.example backend/.env
14 # Editer backend/.env avec vos valeurs
```

```

15
16 # 4. Demarrer les serveurs de developpement
17 npm run dev
18 # Utilise concurrently pour executer :
19 # - Backend : cd backend && nodemon server.js (port 5000)
20 # - Frontend : cd frontend && vite (port 5173)
21 # - Courtier MQTT démarre automatiquement avec le backend sur le port 1883
22
23 # 5. Verifier les services
24 curl http://localhost:5000/api/auth/me # Doit retourner 401 (pas 500)
25 # mqtt-explorer -> connecter sur localhost:1883
26 # Ouvrir http://localhost:5173 # Doit afficher la page d'accueil

```

15.4 Configuration du Firmware ESP32

Listing 15.3 – Constantes de Configuration Firmware (SmartHome_ESP32.ino)

```

1 // Identifiants Wi-Fi
2 const char* WIFI_SSID = "Votre_SSID_WiFi";
3 const char* WIFI_PASSWORD = "Votre_Mot_De_Passe_WiFi";
4
5 // IP du courtier MQTT (votre machine backend sur le reseau local)
6 const char* MQTT_SERVER = "192.168.1.100"; // A modifier
7 const int MQTT_PORT = 1883;
8
9 // Code maison (doit correspondre au codeMaison du compte admin)
10 const char* HOUSE_CODE = "A1234"; // A modifier

```

15.4.1 Installation des Bibliothèques Arduino IDE

Installer les bibliothèques suivantes via **Arduino IDE** → **Croquis** → **Inclure une bibliothèque** → **Gérer les bibliothèques** :

Bibliothèque	Auteur
PubSubClient	Nick O’Leary
ArduinoJson	Benoît Blanchon
DHT sensor library	Adafruit
Adafruit Unified Sensor	Adafruit
ESP32Servo	Kevin Harrington

15.4.2 Étapes de Flashage

1. Connecter l'ESP32 à l'ordinateur via câble USB-C/micro-USB.
2. Dans Arduino IDE : Outils → Carte → ESP32 Arduino → ESP32 Dev Module.
3. Outils → Port → Sélectionner le port COM correct.
4. Mettre à jour le SSID Wi-Fi, le mot de passe, l'IP MQTT et le code maison dans le croquis.
5. Cliquer sur **Téléverser** (Ctrl+U). Attendre le message “Téléversement terminé”.
6. Ouvrir le Moniteur Série (115200 bauds) pour vérifier la connexion Wi-Fi et MQTT.

15.5 Déploiement Production (Render / Railway)

Listing 15.4 – Commande de Démarrage Production

```
1 # Render / Railway : définir la commande de démarrage sur :
2 node index.js
3
4 # Définir toutes les variables d'environnement dans le tableau de bord de la
   ↳ plateforme.
5 # S'assurer que MONGODB_URI pointe vers MongoDB Atlas.
6 # Définir NODE_ENV=production.
7 # HTTPS gère par le reverse proxy de la plateforme.
8
9 # Frontend : deployer sur Vercel ou Netlify
10 # Commande de build : npm run build
11 # Dossier de sortie : frontend/dist
12 # Définir VITE_API_URL=https://votre-backend.onrender.com
```

15.6 Runbook Opérationnel

15.6.1 Vérifications de l'État des Services

Service	Vérification	Résultat Attendu
API Backend	GET /api/auth/me (sans token)	HTTP 401 (pas 500)
Courtier MQTT	Connexion sur le port 1883	Connexion acceptée
MongoDB	Logs de connexion serveur	"Connecté à MongoDB"
Frontend	Charger http ://localhost :5173	Page d'accueil rendue
ESP32	Moniteur Série	"MQTT connected!"

15.6.2 Résolution des Problèmes Courants

MQTT ne se connecte pas (ESP32) : Vérifier que l'IP du courtier correspond à l'IP locale de la machine backend (`ipconfig` sous Windows). S'assurer que les deux appareils sont sur le même réseau Wi-Fi. Vérifier les règles de pare-feu pour le port TCP 1883.

Connexion MongoDB échouée : Vérifier `MONGODB_URI` dans `.env`. Si utilisation d'Atlas, ajouter votre IP à la liste blanche dans les paramètres d'accès réseau Atlas. Vérifier les identifiants.

Erreur de redirection Google OAuth : S'assurer que `http://localhost:5000/api/oauth/google` est ajouté aux URIs de redirection autorisés dans la console Google Cloud.

Urgence gaz ne se lève pas : Vérifier que le compte admin est connecté (pas un compte résident). Seuls les utilisateurs avec le rôle `admin` peuvent appeler `POST /api/gas/clear-emergency`.

DHT11 retourne NaN : Vérifier le câblage : broche DATA sur GPIO 23, VCC sur 3,3V, GND sur GND, résistance de tirage de 10 k Ω entre DATA et VCC. Essayer d'augmenter l'intervalle de lecture.

Annexe D : Matrice de Traçabilité des Exigences

16.1 Objectif

Cette annexe fournit une matrice de traçabilité complète reliant chaque exigence fonctionnelle (du Chapitre 2) à la User Story qui l'implémente, au Sprint dans lequel elle a été livrée, et au cas de test qui la valide.

16.2 Matrice de Traçabilité

Table 16.1 – Matrice de Traçabilité des Exigences

EF-ID	Résumé de l'Exigence	US-ID	Sprint	CT-ID	Validation
EF-A1	Inscription multi-rôles	US-01,02	1	CT-01	Postman : inscrire tous les rôles
EF-A2	Génération code maison unique	US-01	1	CT-01	Vérifier unicité en BD
EF-A3	Rejoindre maison avec code résident	US-02	1	CT-01	Résident inscrit avec code valide
EF-A4	Code secret agence	–	6	–	Postman : inscription agence
EF-A5	Token JWT 30 jours	US-03	1	CT-02	Décoder expiration JWT

EF-ID	Résumé de l'Exigence	US-ID	Sprint	CT-ID	Validation
EF-A6	Hachage mot de passe bcrypt	US-03	1	CT-02	Vérifier BD : pas de texte clair
EF-A7	Google OAuth 2.0	US-04	1	Manuel	Redirection call-back OAuth
EF-A8	TOTP 2FA	US-07	1	Manuel	Vérification Google Authenticator
EF-A9	Vérification email	US-05	1	Manuel	Clic lien email
EF-A10	Réinitialisation mot de passe	US-06	1	Manuel	Flux token email
EF-D1	CRUD Appareils	US-09	2	CT-05	Postman : cycle CRUD complet
EF-D2	Topic MQTT unique	US-09	2	CT-05	Topic dupliqué → 400
EF-D3	20+ types d'appareils	US-09	2	Manuel	Validation enum type appareil
EF-D5	Publication commande MQTT	US-11	2	CT-06	MQTT Explorer + LED physique
EF-D6	Diffusion état Socket.IO	US-17	3	Manuel	Mise à jour second onglet
EF-D7	Restriction appareil résident	US-10	2	CT-04	Résident → 403 mauvaise pièce
EF-F1	Plan d'étage isométrique 3D	US-14	3	Manuel	Vérification visuelle navigateur
EF-F4	Préréglages de scènes	US-16	3	CT-07	Scène Matin + servo
EF-F5	Scène règle luminosité+fenêtre	US-16	3	CT-07	Toutes LEDs + servo vérifiés
EF-S1	ESP32 publication gaz 3s	US-12	2	Manuel	Flux topic MQTT Explorer
EF-S2	Comparaison seuil gaz	US-22	4	CT-08	Test PPM au-dessus du seuil
EF-S3	Activation mode urgence	US-19	4	CT-08	emergencyMode = true en BD
EF-S4	Buzzer + vanne + événement Socket	US-19	4	CT-09	Matériel + alerte navigateur
EF-S5	Levée manuelle admin	US-20	4	CT-09	Endpoint clear, buzzer éteint

EF-ID	Résumé de l'Exigence	US-ID	Sprint	CT-ID	Validation
EF-S6	DHT11 publication 10s	US-13	2	Manuel	MQTT Explorer + vérif BD
EF-IA1	Compagnon IA Groq	US-23	5	CT-10	Réponse commande chat
EF-IA3	IA respecte les permissions	US-23	5	CT-10	Résident refusé mauvaise pièce
EF-SC1	Scénarios temporels	US-24	5	CT-15	Scénario créé via API
EF-SC2	Exécution cron	US-24	5	CT-15	Scène déclenchée à HH :MM
EF-EN1	Calcul énergie	US-25	5	Manuel	Graphique montre kWh non nul
EF-PM1	Demande permission résident	US-29	6	CT-11	Demande créée en BD
EF-PM2	Approbation admin	US-30	6	CT-12	Statut = approuvé, permission ajoutée
EF-PM3	Notification temps réel	US-30	6	CT-12	Événement Socket reçu
EF-AU1	Journalisation d'audit	–	Tous	Manuel	Documents Audit-Log vérifiés
EF-AU2	Vue journal admin	US-28	6	Manuel	Journal filtré rendu
ENF-P2	Limitation débit 200/15min	–	1	CT-13	429 à la 21ème requête auth
ENF-S1	Stockage bcrypt	US-03	1	CT-02	Inspection BD
ENF-U1	Interface responsive	–	Tous	Manuel	Testé à 768px et plus

16.3 Données de Burndown des Sprints

Table 16.2 – Points de Story Restants par Burndown Sprint

Jour	S1	S2	S3	S4	S5	S6
0	34	39	37	36	34	37
2	28	31	29	28	26	29
4	20	23	21	20	18	21
6	14	16	14	13	12	14
8	8	10	8	7	7	8
10	3	4	3	3	3	3
14	0	0	0	0	0	0

16.4 Verrouillage des Versions Technologiques

Table 16.3 – Versions Exactes des Technologies Utilisées

Paquet	Version	Environnement
express	4.19.2	Backend
mongoose	8.3.4	Backend
aedes	0.51.2	Backend
socket.io	4.8.3	Backend
jsonwebtoken	9.0.2	Backend
bcryptjs	2.4.3	Backend
passport	0.7.x	Backend
passport-google-oauth20	2.x	Backend
speakeasy	2.0.0	Backend
qrcode	1.5.4	Backend
groq-sdk	1.1.2	Backend
node-cron	4.2.1	Backend
nodemailer	8.0.7	Backend
helmet	8.1.0	Backend
express-rate-limit	7.x	Backend
express-validator	7.3.2	Backend
react	19.2.4	Frontend
react-router-dom	7.13.2	Frontend
vite	8.0.1	Frontend

Paquet	Version	Environnement
tailwindcss	4.2.2	Frontend
framer-motion	12.38.0	Frontend
three	0.184.0	Frontend
recharts	3.8.1	Frontend
socket.io-client	4.8.3	Frontend
mqtt.js	5.15.1	Frontend
axios	1.14.0	Frontend
jspdf	4.2.1	Frontend
lucide-react	1.7.0	Frontend
Node.js	18+ LTS	Runtime
MongoDB	6.x	Base de données
Arduino IDE	2.x	Firmware
SDK ESP32	2.x	Firmware
PubSubClient	2.8.0	Firmware
ArduinoJson	6.x	Firmware